



Bachelorarbeit

Titel der Arbeit // Title of Thesis

Konzeption und Evaluierung hybrider Echtzeit-Rendering-Pipelines zur selektiven, objektbasierten Stilisierung - Eine empirische Systemanalyse architektonischer Trade-offs von Forward- und Deferred-Ansätzen in komplexen 3D-Szenen

Akademischer Abschlussgrad: Grad, Fachrichtung (Abkürzung) // Degree

Bachelor of Science (B.Sc.)

Autorenname, Geburtsort // Name, Place of Birth

Marvin Baumann, Witten

Studiengang // Course of Study

Medieninformatik

Fachbereich // Department

Informatik und Kommunikation

Erstprüferin/Erstprüfer // First Examiner

Prof. Dr. Gregor Lux

Zweitprüferin/Zweitprüfer // Second Examiner

Prof. Dr. Laura Anderle

Abgabedatum // Date of Submission

Klicken Sie hier, um Text einzugeben.



Eidesstattliche Versicherung

Baumann, Marvin

Name, Vorname // Name, First Name

Ich versichere hiermit an Eides statt, dass ich die vorliegende Abschlussarbeit mit dem Titel

Klicken Sie hier, um Text einzugeben.

selbstständig und ohne unzulässige fremde Hilfe erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate kenntlich gemacht. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Klicken Sie hier, um Text einzugeben.

Ort, Datum, Unterschrift // Place, Date, Signature

Kurzfassung

Abstract Placeholder

Inhaltsverzeichnis

Kurzfassung	I
Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Listings	VI
Abkürzungsverzeichnis	VII
1 Einführung	1
1.1 Motivation & Problemstellung	1
1.2 Ziel der Arbeit & Forschungsfrage	2
1.3 Aufbau der Arbeit	4
2 Grundlagen	5
2.1 Rendering-Paradigmen und algorithmische Komplexität	5
2.1.1 Forward Rendering	5
2.1.2 Deferred Rendering	5
2.2 Hardware-Limitierungen und Performance-Metriken	5
2.2.1 Fragment-Overdraw und Fillrate	6
2.2.2 Speicherbandbreite (Memory Bandwidth)	6
2.2.3 Register Pressure	6
2.2.4 Dynamic Branching und Warp Divergence	6
2.3 Grundlagen der NPR Pipeline	6
3 Stand der Technik und verwandte Arbeiten	7
3.1 Klassische Rendering-Architekturen und deren Implikationen für das NPR	7
3.2 Rendering-Pipelines und Stilisierung in der Industriepraxis	9
3.3 Limitierungen bestehender Ansätze und Identifikation der Forschungslücke	11
4 Konzeption und Realisierung	13
4.1 Konzeption der Systemarchitektur und G-Buffer-Topologie	13
4.2 Mathematisch-Technische Spezifikation der NPR-Komponenten	13
4.2.1 Lokale Shading-Modelle	13
4.2.2 Filter- und Frequenzraum-Operationen	13
4.2.3 Geometriebasierte NPR-Stilisierung	14

4.3	Pipeline-Implementierungen und architektonische Restriktionen	14
4.3.1	Forward Rendering Pipeline	14
4.3.2	Deferred-Uber Pipeline	14
4.3.3	Deferred Light-Volume Pipeline	14
4.4	Telemetrie- und Orchestrator-Architektur	14
4.4.1	Der Render-Orchestrator	15
4.4.2	Die Telemetrie-Infrastruktur	15
4.5	Zusammenfassung	15
5	Evaluationen	16
5.1	Evaluationsmethodik & Experimenten-Design	16
5.1.1	Zielsetzung und Hypothesen der Untersuchung	16
5.1.2	Systemkonfiguration und Variablen-Mapping	18
5.1.3	Versuchsablauf	22
5.2	Geometrische Dichte und Overdraw	22
5.2.1	LOD (Mikro-Geometrie)	23
5.2.2	Objektanzahl	24
5.2.3	Overdraw-Dichte	26
5.3	Bandbreite und Auflösung	27
5.3.1	Auflösung	27
5.3.2	Bandbreiten	28
5.4	Beleuchtung	30
5.4.1	Lichtanzahl	31
5.4.2	Lichtintensität & Overlap	32
5.4.3	Shading Overdraw	35
5.5	Hybride Stilisierung & Entropie	36
5.5.1	Stil-Entropie	36
5.5.2	Kernel Bandbreiten Limits	39
5.6	Macro-Benchmark Synthese	41
6	Diskussion	45
6.1	Zusammenführung und Interpretation der Testergebnisse	45
6.2	Die heuristische Entscheidungsmatrix	45
6.3	Kritische Reflexion und Limitationen	45
6.4	Lessons Learned	46
7	Zusammenfassung und Ausblick	47
7.1	Resümee der Arbeit	47
7.2	Ausblick und zukünftige Arbeiten	47
Literatur		i

Abbildungsverzeichnis

3.1	Speicherlayout der Hi-Fi Rush G-Buffer. Eigene Darstellung in Anlehnung an [13, Folie „Our G-Buffer Layout“].	10
5.1	Geometrische Skalierung bei variierendem Detailgrad	23
5.2	Objektanzahl-Limitierung im CPU-Dispatch	25
5.3	Overdraw-Skalierung bei steigender Tiefenkomplexität	26
5.4	Auflösungsskalierung unter konstanter Beleuchtungslast	28
5.5	Basis-Bandbreitenkosten der Rendering-Architekturen	29
5.6	Basis-Bandbreitenkosten der Rendering-Architekturen	30
5.7	Skalierung der Lichtanzahl bei dynamischen Punktlichtern	32
5.8	Skalierung der Lichtintensität bei konstanter Lichtanzahl	34
5.9	Skalierung der Lichtanzahl in einer Singularität	34
5.10	Entkopplungsnachweis bei steigendem geometrischen Overdraw	36
5.11	Einfluss der räumlichen Entropie auf die GPU-Shading-Zeit	37
5.12	Vergleich der Stil-Entropie Verteilungen in <i>HyDra</i> während der 5.5.1 Test-Suite in Test A (Räumliche Entropie).	38
5.13	Einfluss stilistischer Kombinatorik auf die GPU-Shading-Zeit	38
5.14	Skalierung des Kuwahara-Filterradius	40
5.15	Akkumulierte GPU-Ausführungszeit im Makro-Benchmark	43
5.16	Frame-Zeit-Verlauf des Kamera-Flythroughs	43

Tabellenverzeichnis

5.1	Spezifikation der primären Test-Hardware.	18
5.2	Spezifikation der Software-Umgebung und API-Schnittstellen.	18
5.3	Spezifikation der unabhängigen Variablen, deren formale Definition und Zuweisung zu den Test-Suites.	19
5.4	Spezifikation der abhängigen Variablen (Metriken), deren formale Definition und CSV-Mapping.	20
5.5	Spezifikation der Kontrollvariablen und der statischen System-Baseline. .	21

Listings

Abkürzungsverzeichnis

ALU	Arithmetic Logic Unit	27
API	Application Programming Interface	15
CAD	Computer-Aided Design	1
CPU	Central Processing Unit	13
G-Buffer	Geometry Buffer	2
GPU	Graphics Processing Unit	9
HDR	High Dynamic Range	17
HyDra	Hybrid Deferred Rendering Architecture	3
LOD	Level of Detail	
MRT	Multiple Render Targets	30
MSAA	Multisample Anti-Aliasing	3
MVP	Model-View-Projection	25
NPR	Non-Photorealistic Rendering	1
PBR	Physically Based Rendering	1
RMW	Read-Modify-Write	17
ROP	Raster Operation Pipeline	31
RT	Render Target	9
SIMT	Single Instruction, Multiple Threads	37
TAA	Temporal Anti-Aliasing	8
UBO	Uniform Buffer Object	31
V-Buffer	Visibility Buffer	8
VGPR	Vector General Purpose Register / Vektorregister	39
VRAM	Video Random Access Memory	17

1 Einführung

Die stilisierte Darstellung komplexer 3D-Szenen und einzelner Objekte mittels Non-Photorealistic Rendering (NPR) ist in der modernen Computergrafik sowohl für technische als auch für unterhaltende Anwendungen von großer Bedeutung. Eine gezielte Stilisierung erhöht die visuelle Lesbarkeit, lenkt den Fokus des Betrachters und ermöglicht künstlerische Abstraktionen. Insbesondere die industrielle Praxis und aufwendige Spieleproduktionen sehen sich jedoch mit der Notwendigkeit konfrontiert, stetig wachsende Datenmengen und Szenenkomplexitäten in Echtzeit bewältigen zu müssen. Die effiziente Bewältigung der resultierenden Performance-Engpässe erfordert hochskalierbare Rendering-Architekturen. Die Gewährleistung dieser Skalierbarkeit erweist sich im Kontext der für NPR zwingend erforderlichen Stil-Heterogenität jedoch als komplexe Problemstellung und offenbart einen fundamentalen Konflikt innerhalb etablierter Rendering-Paradigmen. [1, Abschn. 1]

1.1 Motivation & Problemstellung

NPR-Effekte werden primär mit künstlerischen Umsetzungen und stilisierten Darstellungen in der Unterhaltungsindustrie, wie etwa in Videospielen oder Filmproduktionen, in Verbindung gebracht. Die gezielte Stilisierung ganzer Szenen oder einzelner Objekte zur Schaffung eines bestimmten künstlerischen Kontextes bildet hierbei einen der Hauptanwendungszwecke von modernem NPR. Darüber hinaus finden diese Techniken jedoch auch im Bereich von technischen Illustrationen, Computer-Aided Design (CAD)-Anwendungen oder medizinischen Visualisierungen weitreichende Anwendung. In diesen Fachbereichen erfüllt die Stilisierung oftmals den funktionalen Zweck, durch eine simplere oder abstraktere Darstellung die allgemeine Lesbarkeit der Szene zu erhöhen oder den Fokus gezielt auf kritische Daten und spezifische Elemente zu lenken. Eine derartige funktionale Hervorhebung durch Simplifizierung ist mit gängigem *Physically Based Rendering (PBR)* (PBR) in dieser Form oft nicht möglich, da rein fotorealistische Ansätze visuell häufig zu detailreich sind und den Blick auf das Wesentliche verstellen [1, Abschn. 1].

Moderne Echtzeit-Rendering-Pipelines müssen enorme Mengen an 3D-Geometriedaten verarbeiten und simultan hochkomplexe Beleuchtungsszenarien aus zahlreichen Lichtquellen berechnen. Das *Deferred Rendering* entkoppelt die Geometrieverarbeitung strikt von der Lichtauswertung und stellt ein etabliertes Architektur-Paradigma zur effizienten Bewältigung hoher Szenen- und Lichtkomplexität dar, da es in komplexen Szenarien eine

drastische Reduzierung des Rechenaufwands ermöglicht. Aufgrund dieses inhärenten Aufbaus weisen klassische Deferred-Pipelines jedoch einen starken architektonischen Nachteil auf, da sie die darstellbare Materialvielfalt stark limitieren. Infolge dieser technischen Restriktionen wurden Deferred-Paradigmen historisch primär für die Auswertung homogener, fotorealistischer Oberflächen (PBR) konzipiert und optimiert [2, Abschn. 1].

Deferred Rendering projiziert die dreidimensionale Szenengeometrie in einen zweidimensionalen Geometry Buffer (G-Buffer). Bei dieser Überführung in den *Screen-Space* geht die geometrische Identität der Objekte, also semantische Informationen wie etwa die Zugehörigkeit zu einem spezifischen 3D-Modell, weitestgehend verloren. Stattdessen werden in klassischen Pipelines lediglich physikalische Oberflächendaten für standardisierte Lichtberechnungen (PBR), wie etwa Albedo, Normalen oder Rauheit, gespeichert. Viele NPR-Effekte erfordern für eine fehlerfreie Ausführung jedoch zwingend einen gewissen Grad an semantischen Informationen. Die Zuweisung verschiedener Zeichenstile wie *Toon-Shading* oder die exakte Trennung überlappender Objekte durch Umrisslinien benötigen beispielsweise spezifische Material-Identifikatoren (*Shading-Model-IDs*), oder Objekt-IDs. Diese Metadaten fehlen in einem klassischen, auf PBR-optimierten G-Buffer. Dieser Umstand führt dazu, dass NPR in Deferred-Architekturen häufig zu einem rein globalen Post-Processing-Filter über das gesamte Bild verkommt [1, Abschn. 1]. Eine granulare, selektive Stilisierung (z.B. Bauteil A als Skizze, Bauteil B massiv schattiert) wird dadurch extrem erschwert. Bei hoher stilistischer Vielfalt und sehr selektiven Anwendungsfällen wird daher oft auf klassisches *Forward-Rendering* zurückgegriffen. Dieses erweist sich jedoch wiederum als stark ineffizient bei hoher Beleuchtungskomplexität [2, Abschn. 1].

Die Kombination aus massenhafter Geometrie, Beleuchtungskomplexität und stilistischer Vielfalt zwingt in der Praxis gängige Rendering-Paradigmen an ihre individuellen hardware-technischen Leistungsgrenzen. Dabei offenbart sich ein grundlegender architektonischer Konflikt: Pipelines leiden bei der Stilisierung vieler Objekte entweder unter massiven redundanten Zeichenoperationen bei überlappender Geometrie [3, Abschn. 1], oder die Auswertung der diversen Stildaten überlastet die Speicher- und Rechenkapazitäten der Grafikkhardware [4, Abschn. 6]. Trotz der Relevanz dieses Problems mangelt es an systematischen, empirischen Erkenntnissen darüber, unter welchen genauen Bedingungen und Skalierungen diese spezifischen Leistungseinbrüche im Kontext des NPRauftreten. Aus dieser Notwendigkeit leitet sich die direkte Zielsetzung der vorliegenden Arbeit ab.

1.2 Ziel der Arbeit & Forschungsfrage

Das primäre Ziel dieser Arbeit ist die Durchführung einer holistischen, quantitativen Systemanalyse verschiedener Rendering-Architekturen im Kontext der selektiven Stilisierung bzw. NPR, mit einem speziellen Fokus auf das in diesem Bereich oft vernachlässigte Deferred Rendering. Das angestrebte wissenschaftliche Resultat dieser Untersuchung ist

die Erstellung einer empirisch hergeleiteten, heuristischen Entscheidungsmatrix zur Evaluation elementarer Rendering-Paradigmen. Die untersuchten Basis-Pipelines umfassen dabei ein naives Forward Rendering als Referenz, ein simples Deferred Rendering mit einer monolithischen Shader-Struktur sowie einen mehrstufigen Deferred-Ansatz zur volumetrischen Lichtauswertung. Die resultierende Matrix sowie das zugrunde liegende Evaluations-Framework adressieren einen primären ingenieurstechnischen Nutzen: Sie bieten Entwicklern eine objektive Grundlage, um anhand der Variablen Szenenkomplexität und Stil-Entropie die geeignete Pipeline-Architektur zu evaluieren, noch bevor in der Softwareproduktion architektonische Performance-Engpässe entstehen. Darüber hinaus fungiert die entwickelte Architektur als isolierte Referenzumgebung, um das Basisverhalten und die hardwaretechnischen Leistungsgrenzen dieser fundamentalen Paradigmen unabhängig zu quantifizieren.

Aus diesen Zielen leitet sich die zentrale Forschungsfrage der vorliegenden Arbeit ab, die im Rahmen der Systemanalyse empirisch untersucht und beantwortet wird:

Wie lassen sich hybride Non-Photorealistic Rendering-Pipelines architektonisch so konzipieren und umsetzen, dass eine selektive, objektbasierte Stilisierung unter hoher Entropie ermöglicht wird, und wie verhält sich dabei die Gesamteffizienz der Pipeline im Vergleich zur Szenenkomplexität? Welche Aspekte bilden hier die individuellen Performance-Engpässe und Limitierungen und wie verschieben sich diese zwischen den verschiedenen Rendering-Pipelines?

Das C++/OpenGL-basierte Evaluations-Framework Hybrid Deferred Rendering Architecture (HyDra) wird im Rahmen dieser Arbeit als wissenschaftliches Artefakt realisiert, um eine objektive und kontrollierte Testumgebung für die Beantwortung der architektonischen Fragestellungen bereitzustellen. Innerhalb dieses Frameworks werden die drei zuvor genannten Basis-Pipelines isoliert umgesetzt. Dieser methodische Aufbau ermöglicht die Generierung deterministischer Messreihen unter systematischer Skalierung der Variablen der Geometrie, Beleuchtung und Stil-Entropie, ohne den Overhead proprietärer Engine-Architekturen.

Die Implementierung und Evaluation komplexer Sondereffekte, wie etwa die Verarbeitung semitransparenter Geometrie oder hardwarebasiertes Anti-Aliasing (Multisample Anti-Aliasing (MSAA)), sind nicht Teil dieser Untersuchung, da sie den Rahmen der grundlegenden Architekturanalyse sprengen würden. Ebenso werden moderne, auf räumlicher Unterteilung basierende Lichtauswertungen, wie *Clustered*- oder *Forward+*-Rendering, bewusst exkludiert, um sich vollständig und isoliert auf das Basisverhalten der Architekturen unter dem Einfluss hoher Stil-Entropie zu fokussieren.

1.3 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich wie folgt, um eine systematische Beantwortung der Forschungsfragen zu gewährleisten: **Kapitel 2** vermittelt zunächst die architektonischen und hardwaretechnischen Grundlagen sowie relevante technische Begrifflichkeiten, die für das Verständnis der weiteren Untersuchungen erforderlich sind. **Kapitel 3** analysiert im Anschluss den aktuellen Stand der Forschung sowie der Industriepraxis und grenzt darauf aufbauend die bestehende Forschungslücke ab. **Kapitel 4** dokumentiert die Konzeption und softwaretechnische Realisierung des entwickelten Evaluations-Frameworks *HyDra* sowie etwaige praktische und architektonische Grenzen der verschiedenen Rendering-Architekturen. **Kapitel 5** präsentiert die Methodik der durchgeführten Versuchsreihen und wertet die empirischen Messdaten unter den Rendering-Paradigmen aus. **Kapitel 6** und **Kapitel 7** führen die gewonnenen Erkenntnisse abschließend in einer heuristischen Entscheidungsmatrix zusammen, diskutieren die Limitationen der Arbeit und schließen diese mit einem Ausblick auf künftige Forschungsmöglichkeiten und Erweiterungen der Evaluationssoftware ab.

2 Grundlagen

In diesem Kapitel werden die theoretischen Fundamente der untersuchten Rendering-Pipelines definiert. Dazu gehört die Abgrenzung der verschiedenen Render-Pfade, die Spezifikation der physischen Hardware-Flaschenhälse sowie die Einordnung der Konzepte im Kontext der Stilisierung.

2.1 Rendering-Paradigmen und algorithmische Komplexität

Dieser Abschnitt widmet sich der formalen Definition und Gegenüberstellung von klassischen und entkoppelten Render-Pfaden unter Berücksichtigung ihrer inhärenten Skalierbarkeit.

2.1.1 Forward Rendering

Beschreibung der klassischen Render-Pipeline mit der direkten Berechnung von Geometrie und Schattierung in einem Durchlauf. Erläuterung des theoretischen Aufwandsverhaltens bei steigender Objekt- und Lichtanzahl.

2.1.2 Deferred Rendering

Erklärung der entkoppelten Pipeline und der Zwischenspeicherung von Geometriedaten in separaten Puffern. Aufzeigen der Effizienzgewinne bei der Schattierung durch den Einsatz geometrischer Hilfskörper.

2.2 Hardware-Limitierungen und Performance-Metriken

Dieser Abschnitt spezifiziert die physischen Ausführungseinheiten moderner Grafikhardware, welche die kritischen Engpässe bei der praktischen Ausführung der Render-Pfade bilden.

2.2.1 Fragment-Overdraw und Fillrate

Thematisierung mehrfach gezeichneter Pixel auf identischen Koordinaten. Betrachtung der resultierenden Last für die Füllrate und softwareseitiger Optimierungen wie dem Z-Prepass.

2.2.2 Speicherbandbreite (Memory Bandwidth)

Analyse der Durchsatzgrenzen beim Zugriff auf den Videospeicher. Erläuterung des Lese- und Schreibaufwands durch breite Pufferstrukturen und additive Mischvorgänge.

2.2.3 Register Pressure

Behandlung der limitierten internen Speichereinheiten pro Rechenkern. Beschreibung der Auswirkungen auf die Thread-Parallelisierung bei der Nutzung großer, kombinierter Shader-Programme.

2.2.4 Dynamic Branching und Warp Divergence

Erläuterung der synchronen Funktionsweise von Thread-Gruppen auf Hardware-Ebene. Beschreibung der sequenziellen Serialisierung bei ungleichen Verzweigungsbedingungen auf Pixelebene.

2.3 Grundlagen der NPR Pipeline

Dieser Abschnitt kategorisiert die raumbezogenen Konzepte der NPR-Generierung zur selektiven, isolierten Stilisierung von Szenenobjekten.

Abwägung von Stilisierungsverfahren direkt auf der dreidimensionalen Szenengeometrie im Vergleich zu nachgelagerten, globalen Filtern auf dem fertigen zweidimensionalen Bild. Gegenüberstellung von präziser Objektrennung und Laufzeit-Konstanz.

3 Stand der Technik und verwandte Arbeiten

Dieses Kapitel analysiert den aktuellen Forschungsstand zu Forward- und Deferred-Rendering-Architekturen sowie moderne Industriepraktiken im Bereich hybrider Architekturen und selektiver Stilisierung. Durch die Betrachtung bestehender Verfahren im Kontext des NPR werden deren konzeptionelle und leistungstechnische Limitierungen aufgezeigt. Die Identifikation dieser Schwachstellen sowie der Mangel an empirischen Vergleichsdaten dienen der Definition der zugrundeliegenden Forschungslücke, legitimieren die in dieser Arbeit durchgeführte holistische Systemanalyse und ermöglichen die wissenschaftliche Einordnung des eigens entwickelten Evaluations-Frameworks.

3.1 Klassische Rendering-Architekturen und deren Implikationen für das NPR

Deferred-Rendering-Paradigmen stellen sowohl in der akademischen Forschung als auch in der industriellen Praxis etablierte Standards dar [5, S. 2]. Moderne Echtzeit-Rendering-Pipelines werden maßgeblich von komplexen Beleuchtungsanforderungen getrieben, wodurch das effiziente Handhaben von Deferred-Architekturen in den Fokus rückt. Gleichzeitig werden im Bereich des Forward-Renderings durch diverse Optimierungsverfahren die klassischen Schwächen dieses Ansatzes zunehmend relativiert [2], [6]. Die Wahl des Rendering-Ansatzes ist in der Praxis stark kontextabhängig und auf die spezifischen Leistungs- und Qualitätsanforderungen des Zielbildes ausgerichtet. Folglich existieren sowohl Deferred- als auch Forward-Implementierungen in modernen Engines selten in ihrer reinen Form [7, S. 56]. Der wissenschaftliche Diskurs fokussiert sich daher primär auf hybride Lösungsansätze und gezielte Reduktionen architektonischer Engpässe.

Der primäre architektonische Nachteil klassischer Forward-Rendering-Ansätze liegt, wie in Abschnitt 2.1.1 dargelegt, im multiplikativen Aufwand von $O(G \cdot L)$ bei hoher Geometrie- und Beleuchtungskomplexität. Zusätzlich zu dieser Komplexität ergibt sich eine kombinatorische Explosion der Shader-Permutationen, sobald diverse Materialeigenschaften und Lichttypen in einer Szene verrechnet werden müssen. Dieser Limitierung liegt die inhärente Eigenschaft von Forward-Pipelines zugrunde, bei hoher Szenen-Entropie zwangsweise eine gewisse Menge Fragment-Overdraw zu erzeugen [3, S. 1]. Ansätze wie ein Z-Prepass zur frühzeitigen Verwerfung verdeckter Fragmente gehören in Forward-Pipelines zwar zum Standard [7, S. 57], reichen bei hoher Objekt- und Lichtdichte jedoch nicht aus, um die

Effizienz von Deferred-Ansätzen zu erreichen [8, S. 4]. Modernere Techniken wie *Clustered Forward* oder *Forward+* adressieren dieses Problem, indem Lichtquellen in 2D-Kacheln oder 3D-Clustern räumlich sortiert werden [2], [6]. Dies ermöglicht eine Effizienz, die Deferred-Ansätzen nahekkommt, während der primäre Vorteil der Forward-Architektur, die hohe Flexibilität bei der Materialauswertung, erhalten bleibt.

Deferred-Rendering-Pipelines haben sich in Umgebungen mit hoher geometrischer Komplexität und anspruchsvollen Beleuchtungsszenarien weitestgehend durchgesetzt, weisen jedoch ebenfalls diverse architektonische Limitierungen auf. Der signifikanteste Flaschenhals ist die hohe Auslastung der Speicherbandbreite (Memory Bandwidth), welche durch den permanenten Schreib- und Lesezugriff auf den G-Buffer induziert wird [7, S. 55]. Ein moderner Lösungsansatz ist die Implementierung entkoppelter Architekturen wie des Visibility Buffer (V-Buffer) [7], welcher als Shading-Grundlage für hochkomplexe Geometrie-Pipelines wie der *Nanite*-Architektur in der Unreal Engine 5 dient [9, Folie *Visibility Buffer*]. Im Gegensatz zum traditionellen G-Buffer speichert der V-Buffer pro Pixel lediglich minimale geometrische Metadaten (wie Primitive- und Instanz-IDs) sowie den Tiefenwert. Die speicherintensive Materialauswertung wird vollständig in einen nachgelagerten Pass verschoben. Neben der Bandbreitenproblematik existieren in klassischen Deferred-Architekturen zwei weitere Nachteile: die fehlende native Kompatibilität mit hardwarebasiertem Multi-sample Anti-Aliasing (MSAA) sowie die Einschränkung bei der Darstellung transluzenter Oberflächen. Da die Beleuchtung im Screen-Space berechnet wird, sind die für MSAA erforderlichen Geometriedaten zu diesem Zeitpunkt verworfen. Stattdessen müssen temporale Post-Processing-Verfahren (wie Temporal Anti-Aliasing (TAA)) eingesetzt werden [8], [10, S. 5]. Die Evaluierung semitransparenter Geometrie ist im reinen Deferred-Pfad nicht trivial lösbar, da ein Standard G-Buffer pro Pixel nur eine diskrete Tiefenschicht vorhält. Dieses Defizit wird in der industriellen Praxis standardmäßig über einen nachgelagerten hybriden Forward-Pass für transparente Objekte gelöst [7, S. 56].

Hinsichtlich des NPR unterscheidet die aktuelle Literatur primär zwischen *Object-Space*- und *Screen-Space*-Methoden [1, S. 148]. Object-Space-Ansätze sind in der Regel in Forward-Pipelines verankert und berechnen Stilisierungen direkt auf der Geometrie, was eine granulare, objektbasierte Anpassung ermöglicht. Bei hoher Szenenkomplexität weisen diese Verfahren jedoch eine unzureichende Skalierbarkeit auf, was primär auf den Fragment-Overdraw bei rechenintensiven Abstraktionseffekten zurückzuführen ist [3, S. 1]. Die moderne Rendering-Forschung weicht zur Umgehung dieser Performance-Limits oft auf Screen-Space-Methoden (Post-Processing) aus, was architektonisch stark den Paradigmen des Deferred-Renderings entspricht. Wie Magdics et al. [1, S. 148] beschreiben, ist diese Screen-Space-Verarbeitung zwar sehr effizient, löst jedoch den direkten Bezug zum individuellen 3D-Objekt auf, wodurch etwa die selektive Stilisierung stark erschwert wird. Die Nutzung von Deferred-Rendering-Architekturen ermöglicht die Rekonstruktion des räumlichen Kontextes im Post-Processing. Wie Thaler [8, Abschn. 3.2] anmerkt, basiert die Lösung dieses Problems auf einem zweistufigen Ansatz: Im ersten Schritt werden alle räumlichen 3D-Informationen (Tiefen, Normalen) im G-Buffer gesammelt, um diese Metadaten im zweiten

Schritt für globale, bildschirmbasierte NPR-Effekte nutzbar zu machen. Dies ermöglicht es, Stilisierungen im Screen-Space auch geometriebewusst durchzuführen. Klassische Deferred-Pipelines weisen in diesem Kontext jedoch eine fundamentale Schwäche bezüglich der Materialvielfalt auf [2, S. 1]. Das Auswerten zahlreicher spezifischer Materialien oder Stilisierungen in einem vereinheitlichten Screen-Space-Pass ist nur unter starken Performance-Einbußen durch *Dynamic Branching* auf der Graphics Processing Unit (GPU) realisierbar [8, Abschn. 2.6]. In der Konsequenz bleiben performante Stilisierungen bei reinem Deferred-Rendering meist auf globale Post-Processing-Filter beschränkt [1, S. 148].

Zusammenfassend lässt sich festhalten, dass die aktuelle Forschung zwar diverse architektonische Lösungen für hohe Geometrie- und Beleuchtungskomplexität sowohl in Forward- als auch in Deferred-Architekturen bietet, die spezifische Schnittmenge mit selektivem NPR in Deferred-Pipelines jedoch theoretischen Limitierungen unterliegt. Die Analyse der angewandten Industriepraxis sowie der Architektur moderner Game-Engines zeigt auf, wie diese akademischen Hürden in der Praxis umgangen werden.

3.2 Rendering-Pipelines und Stilisierung in der Industriepraxis

Der theoretische Konflikt zwischen der für Deferred Shading charakteristischen Homogenität des Beleuchtungspasses und der für NPR zwingend erforderlichen Heterogenität individueller Stilisierungen zwingt die Industrie, zu architektonischen Kompromissen. Der naive Ansatz, alle für komplexe NPR-Modelle benötigten Parameter separat in den G-Buffer zu schreiben, führt unweigerlich zu einem sogenannten *Fat G-Buffer* [11, S. 71]. In modernen AAA-Produktionen der Spieleindustrie werden teilweise bis zu acht Render Targets (RTs) gleichzeitig beschrieben, was starke Bandbreiten-Engpässe (Memory Bandwidth Bottlenecks) verursachen kann [12, S. 884]. Folglich existiert in der Industriepraxis kein universeller, beziehungsweise einheitlicher, Deferred-NPR-Algorithmus. Stattdessen greifen Entwickler auf hoch spezialisierte, kompromissbehaftete Architekturen zurück, um selektive Stilisierungen innerhalb gegebener Hardware-Grenzen zu realisieren.

Ein primärer Lösungsansatz hierfür ist das *G-Buffer Packing* in Kombination mit „Shading-Model-IDs“. Die Wiederverwendung oder Neudefinition von Datenkanälen, die für das klassische PBR vorgesehen sind, reduziert den Bedarf an Speicherbandbreite. Ein exemplarisches Vorgehen zeigt das auf Toon-Rendering fokussierte Spiel *Hi-Fi RUSH*: Hierbei werden regulär für PBR-Daten vorgesehene G-Buffer-Kanäle gezielt umgewidmet, um diese stattdessen mit spezifischen NPR-Parametern und ID-Masken zu belegen (siehe Abbildung 3.1).

Ein ähnliches Verfahren zeigt die Architektur von *Uncharted 4*, in der eine komprimierte *Material-ID* im G-Buffer mitgeführt wird, um im nachgelagerten Lighting-Pass zwischen

Hi-Fi RUSH G-Buffer Layout

	GBufferA R8G8B8A8		GBufferC R8G8B8A8		GBufferB R8G8B8A8
R	World Normal X	R	BaseColor R	R	Metallic (Fix at 0)
G	World Normal Y	G	Base Color G	G	Specular (Customize)
B	World Normal Z	B	Base Color B	B	Roughness
A	PerObjectGBufferData	A	GBufferAO (Customize)	A	ShadingModelID+SelectiveOutputMask

* Angepasste GBufferAO- und Specular-Kanäle für Toon-Rendering

Abbildung 3.1: Speicherlayout der Hi-Fi Rush G-Buffer. Eigene Darstellung in Anlehnung an [13, Folie „Our G-Buffer Layout“].

verschiedenen Shader-Features zu differenzieren [14, Folie „Deferred Pipeline“]. Überträgt man diesen Ansatz auf das NPR, um eine selektive Stilisierung direkt im regulären Lighting-Pass zu verankern, führt dies in der Praxis unweigerlich zu monolithischen Shader-Strukturen, sogenannten Über-Shader [14, Folie „Problems“]. Eine steigende Stil- und Materialvielfalt induziert innerhalb eines solchen Shaders einen massiven Registerdruck (*Register Pressure*) und provoziert durch die notwendigen Fallunterscheidungen zeitgleich ineffiziente *Warp Divergence*. Diese architektonischen Flaschenhälse limitieren die Gesamteffizienz der Rendering-Pipeline bei hoher Szenen-Entropie auf der GPU signifikant [4, Abschn. 6].

Moderne Engines implementieren hybride Architekturen, die eine grundlegende Deferred-Pipeline um einen nachgelagerten Forward-Pass erweitern, um den strukturellen Limitierungen des G-Buffers hinsichtlich der Materialvielfalt zu begegnen. Shading-Operationen, die hochkomplexe Materialbrechungen oder Transparenzen erfordern, werden in einem oder mehreren, an den Deferred-Pass anschließenden, Forward-Rendering-Schritten berechnet und im finalen Framebuffer kombiniert [12, S. 887]. Da dieser Forward-Pass jedoch die ursprünglichen Schwächen bezüglich der Beleuchtungskomplexität erbt, haben sich Techniken wie *Clustered Forward* [6] und *Forward+* [2] für diesen Hybridschritt etabliert. Durch räumliches Frustum- oder Tile-basiertes Light-Culling reduzieren sie die Anzahl der pro Fragment zu berechnenden Lichtquellen drastisch. Die Etablierung dieser Ansätze als Industriestandard zeigt sich beispielhaft an der *id Tech 6* Engine (*DOOM*, 2016): Die Architektur nutzt hybrides Clustered Deferred und Forward Shading um komplexe Effekte und transparente Objekte mit der opaken Basisgeometrie zusammenzuführen, während die Lichtauswertung durch die räumliche 3D-Cluster-Zuweisung performant skaliert [15].

Eine weitere gängige Industriepraxis bildet das selektive Screen-Space Post-Processing. Hierbei wird die Stilisierung nicht auf Materialebene berechnet, sondern als bildbasierter 2D-Filter in der Nachbearbeitung angebracht [12, S. 661]. Die Industrie implementiert Verfahren wie das *Stencil-Masking*, um bei globalen Filtern dennoch eine selektive, objektbasierte Zuweisung zu gewährleisten [11, S. 81]. In klassischen Deferred-Pipelines ist der *Stencil-Buffer* (*Schablonenspeicher*) hardwareseitig meist direkt an den *Depth-Buffer* (*Tiefenpuffer*). gekoppelt [16, Folie „G-Buffer: Our approach“]. Anstatt Identifikationsnummern aufwendig als Farbinformationen in den regulären G-Buffer zu schreiben, markieren die zu stilisierenden Objekte ihre Pixel während des ersten Render-Durchlaufs (Geometrie-Pass)

mit einem spezifischen Referenzwert direkt im Stencil-Buffer. Wird im Anschluss der Post-Processing-Filter über das fertige Bild gelegt, fungiert dieser Stencil-Wert als Filterkriterium. Die Grafikkarte prüft durch einen fest integrierten Hardware-Test, ob das aktuell berechnete Pixel zu einem markierten Objekt gehört. Ist dies nicht der Fall, wird das Fragment sofort verworfen. Der Shader-Code für den Post-Processing-Effekt kommt somit nur genau dort zur Ausführung, wo er auch tatsächlich benötigt wird. Die *CryEngine 3* demonstriert diese Praxis anschaulich: Sie nutzt Stencil-Masking gezielt, um rechenintensive Post-Processing-Algorithmen wie die Kantenerkennung (Edge-Detection) performant nur auf bestimmte Objekte im Vordergrund anzuwenden, während der Rest der Szene unberührt bleibt [11, S. 81].

Die Betrachtung der Industriepraxis offenbart, dass die Umsetzung selektiver, objektbasierter Stilisierungen in Echtzeit-Engines gegenwärtig aus einem hochkomplexen Patchwork aus G-Buffer-Manipulation, Uber-Shader-Branching und hybriden Forward-Overlays besteht. Dass moderne Rendering-Pipelines hierbei zunehmend fragmentieren, veranschaulicht die Analyse der aktuellen AAA-Produktion *PRAGMATA* (2026), deren selektives Post-Processing durch die Aufspaltung in dutzende isolierte Render-Pässe und eigens generierte Masken realisiert wird [17, Abschn. „Frame“]. Während solche architektonischen Kompromisse für spezifische, isolierten Anwendungsfälle praktikabel sind, skalieren sie nur schwer für Szenen mit extrem hoher Material- und Stil-Entropie. Die Kombination aus steigendem Registerdruck, Bandbreitenlimits und der Fragmentierung in mehrere Rendering-Pässe wirft die fundamentale Frage auf, an welchem Punkt die zugrundeliegenden Rendering-Architekturen performance-technisch Limits aufweisen. Aus dieser Diskrepanz zwischen hoch-spezialisierten Industrie-Workarounds und der Notwendigkeit einer skalierbaren Lösung ergibt sich die Forschungslücke der vorliegenden Arbeit, welche im folgenden systematisch eingegrenzt wird.

3.3 Limitierungen bestehender Ansätze und Identifikation der Forschungslücke

Wie in den Kapiteln 3.1 und 3.2 dargelegt, bieten sowohl die Forschung als auch die Industriepraxis diverse, hochspezialisierte Lösungsansätze für architekturbedingte Restriktionen klassischer Forward- und Deferred-Pipelines, insbesondere im Kontext der selektiven Stilisierung bei NPR. Da eine vollumfängliche Analyse sämtlicher industriespezifischer Workarounds den Untersuchungsumfang dieser Arbeit überschreiten würde, adressiert die vorliegende Untersuchung die bestehende Forschungslücke durch eine empirische Rückbesinnung auf die architektonischen Fundamente. Die Evaluation der systematischen Verschiebung von Performance-Engpässen erfolgt über den Vergleich primärer Rendering-Paradigmen: naives Forward Rendering, simples Deferred Shading mit *G-Buffer-Packing*

über *Style-IDs* und *Uber-Shader* sowie *Deferred Light-Volumes* als Referenz für *Multi-Pass-Architekturen*. Ziel ist die Erarbeitung einer empirischen Basis für eine fundierte Entscheidungsmatrix.

Globale *Screen-Space-Filter* stellen in der Praxis einen einfachen und etablierten Standard dar [1, S. 147]. Im spezifischen Kontext der selektiven, objektbasierten Stilisierung, vornehmlich innerhalb der weit verbreiteten *Deferred-Rendering-Paradigmen*, stoßen gängige Praktiken jedoch in Abhängigkeit von der intendierten visuellen Stilistik an grundlegende architektonische Grenzen. Industrie-Workarounds wie *Stencil-Maskierungen*, dediziertes *G-Buffer-Packing* oder der Einsatz hybrider Pipelines versprechen hierbei zwar Abhilfe, sind jedoch aufgrund ihrer hochgradig spezialisierten Implementierung kaum hinsichtlich ihrer zugrundeliegenden Skalierungsmuster dokumentiert. Es bleibt folglich unzureichend geklärt, welche Szenenkonfigurationen welche spezifischen Lösungsansätze erfordern und an welchen Stellen sich die primären Performance-Engpässe manifestieren.

Theoretische Performance-Engpässe, wie etwa Bandbreitenlimitierungen in *G-Buffern* oder *Register Pressure* bei *Uber-Shader*, sind in der Literatur zwar dokumentiert [4], [12, Abschn. 6]; wie Akenine-Möller et al. [12, S. 905] jedoch anmerken, verschieben sich diese Limitierungen in der Praxis sehr dynamisch in Abhängigkeit von der Szenenkomplexität. Der aktuellen Literatur mangelt es folglich an einer isolierten, komparativen und empirischen Systemanalyse, welche diese Engpässe spezifisch im Kontext des NPR quantifiziert.

Die Gewährleistung valider empirischer Messungen erfordert eine strikte Limitierung des Untersuchungsraums. Komplexe Sondereffekte oder Optimierungen wie Transparenzen, moderne *Clustered-Techniken* oder *Anti-Aliasing-Ansätze* werden für diese fundamentale Basis-Analyse bewusst exkludiert. Der Fokus liegt isoliert auf der Wechselwirkung zwischen opaker Geometrie, dynamischer Beleuchtung und materialbasierter NPR-Shader-Verarbeitung.

Die Schließung dieser Forschungslücke erfordert ein isoliertes Messinstrument, das die zugrundeliegenden *Rendering-Architekturen* anhand ihrer reinen Basis-Paradigmen und frei von proprietärem Engine-Overhead evaluiert. Das im Rahmen der vorliegenden Arbeit entwickelte Evaluations-Framework *HyDra* dient der methodischen Umsetzung dieses Vorhabens. Durch die Erhebung granularer Rohdaten unter skalierender Stil-Entropie und Szenenkomplexität soll eine heuristische Entscheidungsmatrix abgeleitet werden. Die technische Konzeption sowie die architektonischen Spezifika dieses Frameworks bilden den Schwerpunkt des nachfolgenden Kapitels.

4 Konzeption und Realisierung

Dieses Kapitel beschreibt den Aufbau und die technische Umsetzung des Test-Frameworks. Es zeigt die Struktur der Daten, die Grundlagen der Zeicheneffekte sowie die Umsetzung der drei Render-Wege und der Leistungsmessung.

4.1 Konzeption der Systemarchitektur und G-Buffer-Topologie

Übersicht über den Weg der Daten von der Central Processing Unit (CPU) zur Grafikkarte und die Aufteilung des Zwischenspeichers. Tabellarische Auflistung aller Puffer mit ihren genauen Texturformaten. Begründung für die Wahl einer höheren Speichergenauigkeit, um Bildfehler und Ungenauigkeiten beim Zusammenrechnen der Pixel zu verhindern.

4.2 Mathematisch-Technische Spezifikation der NPR-Komponenten

Beschreibung der einzelnen Zeicheneffekte unabhängig von den eigentlichen Render-Wege, damit sich Erklärungen später nicht im Text wiederholen.

4.2.1 Lokale Shading-Modelle

Erklärung von Toon-Shading für harte Farbstufen und von Gooch-Shading für technische Farbverläufe auf Objektoberflächen.

4.2.2 Filter- und Frequenzraum-Operationen

Erklärung von Rechenmatrizen zur Erkennung von Kanten im Tiefenbild und im Richtungs-bild der Oberflächen. Beschreibung des Algorithmus zur Bildglättung ohne das Verwischen wichtiger Strukturen.

4.2.3 Geometriebasierte NPR-Stilisierung

Technische Erklärung der Außenlinien-Erzeugung durch das Aufblähen der Geometrie entlang ihrer Eckpunkte und das anschließende Ausblenden verdeckter Seiten.

4.3 Pipeline-Implementierungen und architektonische Restriktionen

Erklärung der drei gebauten Render-Wege und Beschreibung der jeweiligen technischen Grenzen für die spätere Auswertung.

4.3.1 Forward Rendering Pipeline

Umsetzung des klassischen Render-Wegs. Erklärung, warum sich globale Bildeffekte hier nicht ohne Weiteres für einzelne Objekte nutzen lassen. Aufzeigen von Hardware-Größenlimits beim Übertragen der Lichtdaten von der CPU.

4.3.2 Deferred-Über Pipeline

Trennung von Geometrie und Schattierung über ein einfaches Vollbild-Viereck. Erklärung, warum bestimmte Filter zwingend erst ganz am Ende auf das fertige Bild angewendet werden können, und wie sich ein großer Shader negativ auf die parallele Ausführung auswirkt.

4.3.3 Deferred Light-Volume Pipeline

Nutzung von einfachen Hilfskörpern zur räumlichen Begrenzung des Lichts. Nachweis von Vorteilen bei der getrennten Berechnung von Effekten pro Objekt und Analyse von Fehlern bei weit entfernten oder sehr kleinen Objekten.

4.4 Telemetrie- und Orchestrator-Architektur

Beschreibung des Codes zur Steuerung des Programms und zur genauen Messung der Ausführungszeiten.

4.4.1 Der Render-Orchestrator

Zentraler Code-Teil zur Verwaltung des Programmablaufs und für das Umschalten zwischen den drei Render-Wegen während der Laufzeit.

4.4.2 Die Telemetrie-Infrastruktur

Umsetzung von asynchronen Application Programming Interface (API)-Abfragen zur Zeitmessung auf der Grafikkarte, ohne das System künstlich auszubremsten. Architektur zur zusätzlichen Erfassung von Grafikspeicher-Auslastung und Befehls-Zählern.

4.5 Zusammenfassung

Kurzer Rückblick auf das gebaute System und Überleitung zur praktischen Überprüfung der genannten technischen Grenzen im nächsten Kapitel.

5 Evaluationen

Nach der Definition der softwaretechnischen Realisierung und dem Aufzeigen erster Grenzen und Eigenheiten der untersuchten Rendering-Paradigmen in Kapitel 4 werden die implementierten Architekturen isolierten Stresstests unterzogen. Das primäre Ziel dieses Kapitels liegt in der objektiven Quantifizierung der zuvor theoretisch beschriebenen Performance-Engpässe unter einer systematischen Skalierung von Szenenkomplexität und Stil-Entropie. Der erste Abschnitt definiert die methodischen Rahmenbedingungen sowie die technischen Spezifikationen des Versuchsaufbaus, um eine belastbare Beantwortung der Forschungsfragen zu gewährleisten. Darauf aufbauend erfolgt die schrittweise Datenauswertung anhand spezifischer Kontrollgruppen und der gezielten Untersuchung vermuteter Performance-Engpässe, bevor ein abschließender Macro-Benchmark die gewonnenen Erkenntnisse ganzheitlich evaluiert.

5.1 Evaluationsmethodik & Experimenten-Design

Das in Kapitel 4 konzipierte Test-Framework fungiert im Folgenden als Messwerkzeug für die empirische Analyse der darin implementierten Rendering-Pipelines. Für die objektive und isolierte Betrachtung der verschiedenen postulierten Performance-Engpässe definiert der vorliegende Abschnitt die methodischen Grundlagen. Dies umfasst die Formulierung der Hypothesen selbst, die exakte Spezifikation des Test-Systems und des Variablen-Mappings sowie die Orchestrierung des Versuchsablaufs.

5.1.1 Zielsetzung und Hypothesen der Untersuchung

Wie in Kapitel 3 dargelegt, weisen die theoretischen Architekturparadigmen des naiven Forward-, Deferred-Uber- und Deferred-Volume-Renderings spezifische Leistungsmerkmale auf. Das primäre Ziel der nachfolgenden Testreihen besteht in der isolierten Quantifizierung dieser technologischen Trade-offs. Es sollen die individuellen Kipppunkte auf Hardware-Ebene lokalisiert und evaluiert werden. Dadurch soll vermieden werden, eine Pipeline den anderen gegenüber als inhärent überlegen zu präsentieren.

Auf Basis der in Kapitel 3 dargelegten theoretischen Faktenlage werden für die implementierten Rendering-Pipelines spezifische Leistungshypothesen formuliert. Diese Annahmen

gilt es im Folgenden durch gezielte Evaluationen objektiv zu verifizieren oder zu falsifizieren. Die empirische Untersuchung beleuchtet dabei gleichzeitig die hardwaretechnischen Grenzen und strukturellen Schwächen des verwendeten Evaluations-Frameworks *HyDra*.

Für die als Referenz gewählte, naive Forward-Rendering-Pipeline werden zwei primäre Leistungsengpässe postuliert. Auf GPU-Ebene wird bei steigender Szenen- und Beleuchtungskomplexität erwartet, dass die Kombination aus geometrischem Fragment-Overdraw und der multiplikativen algorithmischen Komplexität der direkten Beleuchtungsberechnung von $O(\text{Fragmente} \cdot \text{Lichter})$ zu signifikanten Leistungseinbußen führt (**Hypothese H1.1**). Hinsichtlich der CPU-Auslastung wird angenommen, dass das CPU-seitige Filtern und Zuordnen von Lichtquellen pro Objekt sowie das hochfrequente Binden objektspezifischer Licht-Uniforms einen signifikanten API-Overhead provoziert, der die CPU-Zuweisungszeit bei anwachsender Instanzenanzahl nicht-linear skalieren lässt (**Hypothese H1.2**).

Für die klassische Deferred-Pipeline mit einfachem Beleuchtungs-Pass monolithischem Uber-Shader wird postuliert, dass der signifikanteste Leistungsengpass bei einer hohen räumlichen Stil-Entropie auftritt (**Hypothese H2**). Wie in den vorangegangenen Kapiteln theoretisch hergeleitet, erzwingen feingranular gestreute NPR-Effekte divergente Code-Zweige. Dies führt auf GPU-Ebene unweigerlich zu ineffizienter *Warp-Divergence* (siehe 3.2). Begleitet wird dieses Phänomen von einem anwachsenden Registerdruck (*Register Pressure*) bei erhöhter Stil-Vielfalt. Diese kombinierten Hardware-Flaschenhälse zwingen die Architektur zu einer sequenziellen Abarbeitung der Pixel und lassen eine messbare Degradation der Shading-Performance bei stark vermischten Effekten erwarten.

Für die Deferred-Pipeline auf Basis von Light-Volumes postuliert **Hypothese H3**, dass diese Architektur eine hohe Resilienz gegenüber räumlicher Stil-Entropie aufweist und die Branching-Problematik des Uber-Shaders umgeht. Der primäre Leistungsengpass dieser Methode wird stattdessen in der Sättigung der Speicherbandbreite vermutet. Das additive High Dynamic Range (HDR)-Blending bei einer hohen Anzahl überlappender Light-Volumes provoziert überproportional viele Read-Modify-Write (RMW)-Zyklen im Framebuffer (Video Random Access Memory (VRAM)). Der Uber-Shader umgeht diesen spezifische Speicher-Bottleneck konstruktionsbedingt.

Im vorliegenden Kapitel werden die untersuchten Limitierungen ausschließlich auf Hardware-Leistungs-Ebene betrachtet. Spezifische harte Implementierungsgrenzen der Pipelines (wie etwa die Uniform-Buffer-Limitierungen der Forward- und Deferred-Uber-Architekturen) sowie etwaige künstlerische Qualitätsengpässe fließen nicht in die Messreihen ein. Diese Aspekte wurden bereits in Kapitel 4.3 verortet und evaluiert. Da weiterhin im Rahmen dieser Makro-Evaluation keine herstellereinspezifischen Hardware-Profiler wie *NVIDIA Nsight* eingesetzt wird, um Mikrometriken wie exakte Cache-Hit-Raten oder Thread-Maskierungen auf Siliziumebene auszulesen, werden die architektonischen Engpässe deduktiv aus den gemessenen Rendering-Laufzeiten abgeleitet. Die benannten Ursachen sind daher stark korrelierende, theoretische Erklärungsmodelle für das beobachtete Performance-Verhalten.

5.1.2 Systemkonfiguration und Variablen-Mapping

Im Folgenden werden die Referenzdaten der Testumgebung und des verwendeten Software-Stacks beschrieben. Weiterhin werden die für die Testreihen angewendeten unabhängigen Variablen, abhängigen Variablen (Metriken) sowie die entsprechenden Kontrollvariablen dargelegt und definiert.

Die in Tabelle 5.1 aufgezeigten Hardware-Spezifikationen bestimmen die absoluten physikalischen Leistungsgrenzen des Systems. Die in Tabelle 5.2 dokumentierten Software-Schnittstellen und Treiber-Versionen definieren die CPU-seitige Code-Optimierung sowie den grundlegenden API-Overhead.

Tabelle 5.1: Spezifikation der primären Test-Hardware.

Hardware-Komponente	Spezifikation / Wert
Prozessor (CPU)	Intel Core i5-6600K @ 4 × 3.50 GHz
Hauptspeicher (RAM)	16 GB DDR4 (Dual-Channel) @ 3000 MT/s
Grafikkarte (GPU)	NVIDIA GeForce GTX 1660
GPU-Architektur	Turing (TU116)
VRAM-Kapazität	6 GB GDDR5
Theoretische VRAM-Bandbreite	192,1 GB/s
Raster Operation Pipelines (ROPs)	48

Tabelle 5.2: Spezifikation der Software-Umgebung und API-Schnittstellen.

Software / API	Version
Betriebssystem	Linux Mint 22.3 (Kernel 6.8.0)
Grafiktreiber	NVIDIA 595.71.05
C++ Compiler	GCC 13.3.0
Anwendungs-Framework	raylib 5.5
Grafik-API	OpenGL 3.3.0 Core Profile
Shading Language	GLSL 3.30

Nach der Definition der Hardware- und Software-Spezifikationen erfordert das Evaluation-Design eine exakte Abgrenzung der unabhängigen Variablen. Diese Parameter manipuliert das Framework während der Testreihen systematisch. Es sollen Leistungsgrenzen der Hardware isoliert provoziert werden. Tabelle 5.3 fasst das Variablen-Mapping zusammen.

Im direkten Gegenzug zu den manipulierten Parametern erfasst das Telemetrie-Modul des Frameworks eine Reihe quantitativer Messgrößen. Diese abhängigen Variablen bilden die empirische Datengrundlage der Evaluationen. Die Aggregation der Daten pro Frame sowie deren tabellarische Aufarbeitung dienen der isolierten Beurteilung der Hardware- und

Tabelle 5.3: Spezifikation der unabhängigen Variablen, deren formale Definition und Zuweisung zu den Test-Suites.

Variable (Symbol & CSV-Mapping)	Wertebereich	Kapitel	Beschreibung
Render-Architektur (A) activeRenderPath	<i>Enum</i>	Alle	Die primäre architektonische Variable. Rotiert durch Forward, Deferred-Uber und Deferred-Volume.
Geometrischer Detailgrad (LOD) currentLodIndex	12 bis 8.192 Dreiecke	5.2.1	Skaliert die Dreiecksanzahl pro zu renderndem Objekt.
Instanzen-Anzahl (N_{Inst}) activeObstacleCount	1.000 bis 50.000	5.2.2, 5.2.3	Anzahl der gerenderten Geometrie-Instanzen.
Framebuffer-Auflösung (R) renderResolution	480p bis 4K	5.3.1	Absolute Render-Auflösung.
Screen-Space-Abdeckung (C_{Floor}) renderFloor	<i>Boolean</i>	5.3.2	Erzeugt eine Bildfüllende Geometrie.
Lichtquellen-Anzahl (N_{Light}) activeLightCount	10 bis 5.000	5.4.1, 5.4.2	Anzahl der aktiven dynamischen Punktlichter im View-Frustum.
Beleuchtungs-Radius (r_{Light}) lightIntensity	0,5 bis 20,0	5.4.2	Skaliert das Proxy-Volumen der Lichter.
Räumliche Entropie ($E_{Spatial}$) useClusteredStyles	<i>Boolean</i>	5.5.1	Steuert die Verteilung der Stile (geklustert vs. gestreut).
Stil-Kombinatorik (S_{Count}) enableGooch/Toon	1 bis 3 Stile	5.5.2	Skaliert die Anzahl der aktiv evaluierten Shading-Modelle.
Filter-Radius ($r_{Kuwahara}$) kuwaharaRadius	2 bis 16	5.5.3	Ausdehnung des Post-Processing-Kernels.
Makro-Pass (P_{Macro}) Feature Accumulation	Pass 1 bis 4	5.6	Kumulative Aktivierung von Geometrie, Shading und Stil-Entropie über einen fixierten Kamerapfad.

Softwareleistung für die spätere statistische Auswertung. Tabelle 5.4 beschreibt die extrahierten Leistungs- und Szenenmetriken. Die GPU-Laufzeiten werden dabei über asynchrone Hardware-Queries ermittelt, um ein Blockieren der CPU-Pipeline zu verhindern.

Tabelle 5.4: Spezifikation der abhängigen Variablen (Metriken), deren formale Definition und CSV-Mapping.

Metrik (Symbol & CSV-Mapping)	Kategorie	Beschreibung
Logik-Laufzeit (t_{Logic}) cpuLogicMs	CPU	Dauer des logischen Szenen-Passes (Frustum Culling, Sichtbarkeitsprüfung und Matrix-Updates).
Dispatch-Laufzeit (t_{Dispatch}) cpuRenderMs	CPU	Dauer der API-Befehlsübermittlung an den Grafiktreiber (<i>Draw-Call Submission</i>).
Geometrie-Laufzeit (t_{Geom}) geomMs	GPU	Dauer des asynchronen Hardware-Queries für den Geometrie-Pass (G-Buffer-Generierung, entfällt bei Forward).
Beleuchtungs-Laufzeit (t_{Light}) lightMs	GPU	Dauer des asynchronen Hardware-Queries für Beleuchtung und NPR-Berechnung (bei Forward inkludiert dies zwangsläufig die Geometrielast).
GPU-Gesamtlaufzeit (t_{GPU}) totalGpuMs	GPU	Aggregierte GPU-Ausführungszeit des gesamten Frames, gemessen über Hardware-Timer-Queries.
System-Frame-Time (t_{Frame}) totalFrameTimeMs	System	Absolute Frame-Latenz (Delta Time), gemessen über CPU-High-Resolution-Timer (std::chrono).
Sichtbare Instanzen (N_{Vis}) activeInstances	Szene	Anzahl der verarbeiteten Objekt-Instanzen nach dem Frustum Culling.
Gerenderte Dreiecke (N_{Tris}) activeTris	Szene	Berechnete Summe der an die GPU übermittelten Dreiecke ($N_{\text{Vis}} \times \text{Dreiecke des aktiven LODs}$).
Overdraw-Faktor (O_{Draw}) currentOverdraw	Szene	Theoretischer Überzeichnungsfaktor, basierend auf Frustum-Tiefe und räumlicher Instanzdichte.

Den Abschluss des Variablen-Mappings bilden die Kontrollvariablen. Diese Parameter definieren die statische Baseline des Frameworks. Diese Werte bleiben konstant, um reproduzierbare Bedingungen für alle Messreihen zu gewährleisten, es sei denn, die jeweilige Test-Suite überschreibt sie explizit als unabhängige Variable. Tabelle 5.5 fasst die Standardkonfiguration der Kontrollvariablen zusammen. Besonders kritisch für die Validität der Messungen ist die definierte Warm-up-Phase in Echtzeit.

Tabelle 5.5: Spezifikation der Kontrollvariablen und der statischen System-Baseline.

Kategorie	Variable (Symbol & Mapping)	Standardwert (Baseline)
Rendering & Pipeline	HDR-Genauigkeit (HDR_{16bit}) use16BitHDR	Aktiviert (Basis für RMW-Blending)
	Basis-Auflösung (R_{Base}) renderResolution	1920 × 1080 Pixel (1080p)
	Screen-Space-Abdeckung (C_{Floor}) renderFloor	Aktiviert
Verteilung & Geometrie	Basis-Instanzen (N_{Inst_Base}) activeObstacleCount	5.000 Objekte
	Spawn-Radius (r_{Spawn}) objectSphereRadius	200,0 Einheiten
	Basis-Detailgrad (LOD_{Base}) currentLodIndex	1 (Zweithöchste Detailstufe)
	Räumliche Entropie ($E_{Spatial}$) useClusteredStyles	Deaktiviert (Gestreute Verteilung)
Licht-Physik	Umgebungslicht ($I_{Ambient}$) ambientLightStrength	1,0 (Neutrale Grundausleuchtung)
	Licht-Radius (r_{Light}) lightIntensity	2,0 Einheiten
	Licht-Singularität (S_{Light}) useLightSingularity	Deaktiviert (Zufällige Verteilung)
NPR / Stilisierung	Aktive Stile (S_{Count}) enableGooch/Toon	Deaktiviert (0 Stile, reine Basis-Beleuchtung)
	Post-Processing (P_{FX}) enableOutlines/Kuwahara	Deaktiviert
	Filter-Radius ($r_{Kuwahara}$) kuwaharaRadius	4 Pixel
	Filter-Intensität ($i_{Kuwahara}$) kuwaharaIntensity	4,0
Test-Orchestrierung	Warm-up-Dauer (T_{Warmup}) warmupDuration	10,0 Sekunden (Cache-Sättigung)
	Sampling-Dauer (T_{Sample}) captureDuration	1.000 Frames (Statistische Mittelwertbildung)

5.1.3 Versuchsablauf

Vor der aktiven Datenaufzeichnung durchläuft das System eine obligatorische Warm-up-Phase von zehn Sekunden. Dieser Prozess dient dazu, die Hardware-Caches und Textur-Caches aufzubauen [12, Kap. 18.4, Abschn. *Memory Issues*], [18]. Hierdurch werden inkonsistente Frametimes, die durch asynchrone Ladevorgänge, späte Speicherallokationen oder aufgeschobene Statusänderungen des Grafiktreibers in den ersten Frames entstehen [12, Kap. 18.4.2, Abschn. *State Changes*], für die Messung exkludiert. Beim Wechseln von Rendering-Pipelines oder beim Voranschreiten abhängiger Schritt-Variablen wird ebenfalls eine Warm-up-Pause von zehn Sekunden erzwungen.

Nach der Warm-up-Phase startet die Sampling-Phase mit einer festgelegten Dauer von 1000 Frames. Die Kamera wird eingefroren (Ausnahme: Evaluation 5.6). Das Telemetrie-Modul fragt nun jede der definierten abhängigen Variablen pro Frame ab und akkumuliert die Rohdaten in dedizierten Arrays zur späteren Exportierung.

Die final präsentierten Graphen und Messergebnisse basieren auf arithmetischen Mittelwerten der Sampling-Phase. Identifikation und Bereinigung unvorhersehbarer Micro-Stutterer (*Frame-Rate Spikes*) erfolgt durch die Standardabweichung der Frame-Zeiten [19, Abschn. 8.5.2.2]. Für jede Parameter-Konfiguration bildet die Menge der erfassten Frame-Metriken den initialen Datensatz X . Aus diesem werden gruppenbasiert der empirische Mittelwert μ sowie die Standardabweichung σ berechnet. Ein individueller Messwert $x_i \in X$ wird ausschließlich dann in die finale Kurvengenerierung einbezogen, sofern die folgende Bedingung erfüllt ist:

$$\mu - 3\sigma \leq x_i \leq \mu + 3\sigma \quad (5.1)$$

Datenpunkte außerhalb dieses Intervalls klassifiziert der Algorithmus als statistische Ausreißer und verwirft diese für die finale Auswertung. Dieser Schritt garantiert, dass die später präsentierten Messdaten das rein architektonische und algorithmische Skalierungsverhalten der Rendering-Pipelines abbilden.

5.2 Geometrische Dichte und Overdraw

Im Rahmen des nachfolgenden Kapitels werden Benchmarks und Erkenntnisse präsentiert, die dazu dienen, geometriebedingte Performance-Bottlenecks innerhalb der untersuchten Rendering-Pipelines zu evaluieren. Der Fokus liegt primär auf der Isolation der Hardware-Rasterizer und der CPU-Draw-Call-Aufwände.

5.2.1 LOD (Mikro-Geometrie)

Das Ziel dieser Testreihe besteht in der Isolation des Hardware-Rasterizers zur Evaluierung der geometrischen Basis-Last. Konkret soll das absolute Limit der Vertex-Verarbeitung (Triangle Throughput) identifiziert und der Einfluss des G-Buffer-Speicherdurchsatzes isoliert bewertet werden. Zu diesem Zweck wird die Pipeline einem reinen Geometrie-Stresstest ohne Beleuchtungseinflüsse ($N_{\text{Light}} = 0$) ausgesetzt, um zu prüfen, ob die massiven Speicherschreibvorgänge der Deferred-Pipelines bei hoher Dreiecks-Dichte im Vergleich zur Forward-Pipeline einen signifikanten Bandbreiten-Engpass erzeugen [7, S. 55].

Versuchs-Setup

Unabhängige Variable:	Geometrischer Detailgrad ($LOD \in \{0, \dots, 4\}$)
Skalierung (Dreiecke):	24.000 bis 16,38 Mio. Dreiecke
Kontrollvariablen:	$N_{\text{Inst}} = 2.000$, $N_{\text{Light}} = 0$, $C_{\text{Floor}} = \text{Deaktiviert}$, Culling = 0%, Statische Kamera
Zielgrößen:	GPU-Gesamtlaufzeit (t_{GPU}), Dispatch-Laufzeit (t_{CPU})

Wie aus den Ergebnissen in Abbildung 5.1 zu erkennen ist, verhalten sich die verschiedenen Rendering-Pipelines nahezu identisch und skalieren auch bei erhöhtem Detailgrad (LOD) äquivalent, sowohl in Bezug auf die GPU- als auch die CPU-Auslastung. Weiterhin ist zu beobachten, dass sich die GPU-Gesamtlaufzeit (t_{GPU}) nahezu linear zur absoluten Dreiecksanzahl ($O(N)$) verhält, wohingegen die Dispatch-Laufzeit der CPU weitestgehend konstant ($O(1)$) bleibt.

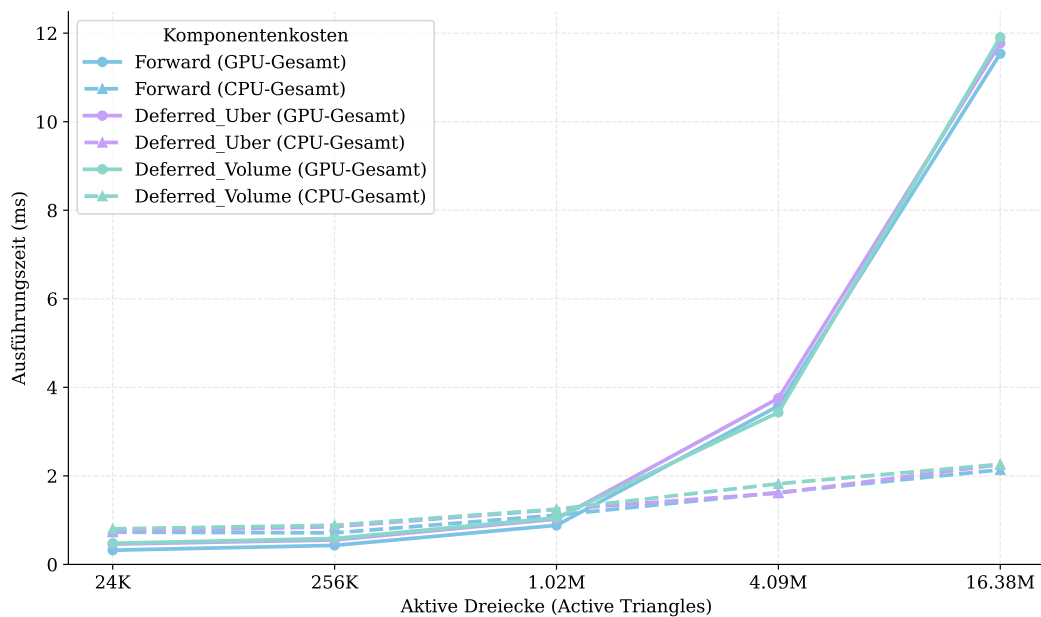


Abbildung 5.1: Messung der GPU- und CPU-Laufzeiten bei einer LOD-Skalierung von 24.000 bis 16,38 Mio. Dreiecken und einer konstanten Objektanzahl von 2.000.

Die ermittelte Laufzeit-Parität zwischen den Pipelines widerlegt zudem die Hypothese der hohen Speicher- und Bandbreitenkosten für reine Geometrieszenarien [12, S. 905]. Das Füllen der breiteren Deferred-Buffer erzeugt gegenüber dem Forward-Buffer keinen limitierenden Bandbreiten-Overhead auf dem VRAM-Bus.

Es sei jedoch methodisch kritisch angemerkt, dass eine rein geometrische Skalierung auf konstantem Raum nur bedingt geeignet ist, um die VRAM-Speicherbandbreite zu sättigen. Die extrem hohe Dreiecksdichte bei $LOD = 4$ (16,38 Mio. Dreiecke auf 2.000 Instanzen bei 1080p) führt dazu, dass die Größe der verarbeiteten Dreiecke den Subpixel-Bereich erreicht oder sogar unterschreitet [3, S. 1]. Die Hardware stößt bei der Transformation dieser winzigen Dreiecke an ihre Kapazitätsgrenze, noch bevor eine signifikante Anzahl von Fragmenten erzeugt werden kann, die den VRAM-Bus überhaupt belasten würden [12, Kap. 18.2.3], [9, Folie *Tiny triangles*].

5.2.2 Objektanzahl

Das Ziel der folgenden Testreihe besteht in der Isolation der CPU-Architektur. In der vorliegenden Untersuchung werden die Draw-Call-Generierung sowie die Ausführungszeit des logischen Szenen-Passes (Frustum Culling, Sichtbarkeitsprüfung und Matrix-Updates) durch eine schrittweise Skalierung der Geometrie-Instanzen evaluiert. Die Durchführung des Tests erfolgt ohne Beleuchtungseinflüsse ($N_{\text{Light}} = 0$) und bei minimalem geometrischen Detailgrad ($LOD = 0$, 12-Dreieck-Würfel), um potenzielle GPU-Engpässe zu eliminieren und eine isolierte Betrachtung der Host-Seite zu gewährleisten. Es wird primär die Hypothese geprüft, ob der CPU-Overhead das Gesamtsystem limitiert, bevor die GPU vollständig ausgelastet ist, und inwiefern architektonische Unterschiede zwischen den Rendering-Pipelines im Dispatch-Prozess auf der Host-Seite (CPU) existieren.

Versuchs-Setup	
Unabhängige Variable:	Instanzen-Anzahl ($N_{\text{Inst}} \in \{1.000, \dots, 50.000\}$)
Dynamische Variable:	Kamera-Distanz (Skaliert proportional zu N_{Inst} zur Vermeidung von Culling-Verzerrungen)
Kontrollvariablen:	$LOD = 0$, $N_{\text{Light}} = 0$, $C_{\text{Floor}} = \text{Deaktiviert}$
Zielgrößen:	Logik-Laufzeit (t_{Logic}), Dispatch-Laufzeit (t_{Dispatch})

Die Ergebnisse in Abbildung 5.2 zeigen, dass sowohl die Laufzeiten des logischen Szenen-Passes (t_{Logic}) als auch die Dispatch-Laufzeiten (t_{Dispatch}) mit der Instanzenanzahl korrelieren, wobei die jeweiligen Steigungsraten signifikant divergieren. Der Anstieg von t_{Dispatch} verläuft zwar linear, verbleibt jedoch selbst bei einer Volllast von 50.000 Instanzen mit ca. 7.8 ms über alle Rendering-Pipelines hinweg stabil und effizient. Im Gegensatz dazu weist t_{Logic} ein deutlich steileres Anstiegsverhalten auf, wobei sich die Forward-Pipeline bei

zunehmender Last zunehmend von den Deferred-Pipelines entkoppelt: In der Traversierung des logischen Szenen-Passes generiert die Forward-Architektur eine messbar höhere CPU-Last als die Deferred-Äquivalente.

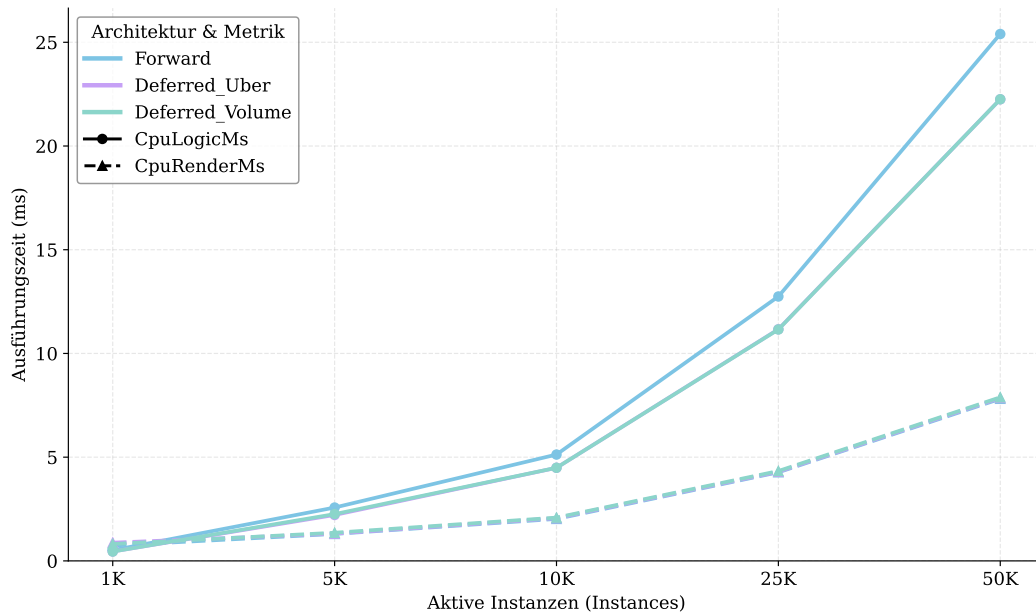


Abbildung 5.2: Messung von Logik-Laufzeit (t_{Logic}) und Dispatch-Laufzeit (t_{Dispatch}) bei einer Skalierung von 1.000 bis 50.000 Instanzen.

Die Analyse ergibt, dass die reine API-Befehlsübermittlung an die Grafikkarte bei keinem der untersuchten Rendering-Paradigmen einen primären Engpass darstellt. Darüber hinaus kann beobachtet werden, dass die umfangreichen mathematischen Operationen (Frustum Culling, Model-View-Projection (MVP)-Matrix-Updates) die Datenverarbeitung der CPU an ihre Grenzen bringt. Das Forward Rendering weist in diesem Kontext eine geringere Skalierbarkeit auf, da es pro Objekt auf der Host-Seite (CPU) eine Lichtliste evaluieren und binden muss [12, S. 891]. Wie Thaler [8, S. 1] beschreibt, müssen bei Single-Pass-Forward-Architekturen, wie sie in dem vorliegenden Framework implementiert sind, für jedes Objekt zunächst alle beeinflussenden Lichtquellen gesucht werden. Dieser Suchlauf kostet kontinuierlich CPU-Zeit, auch wenn er bei $N_{\text{Light}} = 0$ für alle $N_{\text{Inst}} = 50.000$ Objekte vollständig ins Leere läuft. Das schlechtere Skalierungsverhalten der Forward-Pipeline innerhalb der Traversierung des logischen Szenen-Passes verifiziert weiterhin Hypothese H1.2.

Dies stellt gleichzeitig eine architektonische Limitation des genutzten Frameworks dar, da eine alternative Implementierung des Forward Renderings in multiplen Pässen (ein Render-Pass pro Lichtquelle) diese spezifische Suchlogik umgangen hätte.

5.2.3 Overdraw-Dichte

Der folgende Versuch zielt darauf ab, den Einfluss extremer geometrischer Tiefenkomplexität (Overdraw) auf die Speicherbandbreite der GPU zu evaluieren. Zu diesem Zweck wird N_{Inst} auf engstem Raum ($r_{\text{Spawn}} = 30,0$) signifikant erhöht, um eine substantielle Z-Achsen-Stapelung zu erzwingen. Aufgrund der Tatsache, dass Deferred Rendering beim Beschreiben des G-Buffers signifikant mehr VRAM-Bandbreite benötigt als Forward Rendering, wird eine Prüfung durchgeführt, ob ein hoher Overdraw-Faktor (O_{Draw}) die Deferred-Pipelines im Vergleich unverhältnismäßig stark ausbremst (Bandbreiten-Sättigung durch massives Überschreiben von Pixeln).

Versuchs-Setup

Unabhängige Variable:	Instanzen-Anzahl ($N_{\text{Inst}} \in \{2.000, \dots, 32.000\}$)
Metrik (X-Achse):	Theoretischer Overdraw-Faktor ($(\approx 1,1 \text{ bis } 18,1)$)
Kontrollvariablen:	Fixer Radius ($r_{\text{Spawn}} = 30,0$), $LOD = 1$, $N_{\text{Light}} = 0$
Zielgröße:	GPU-Gesamtlaufzeit (t_{GPU})

Obwohl alle analysierten Rendering-Pipelines einen messbaren Anstieg der GPU-Ausführungszeit aufweisen, sind, wie in Abbildung 5.3 zu beobachten, die Abweichungen untereinander minimal. Des Weiteren ist festzustellen, dass alle Rendering-Pipelines nahezu linear skalieren.

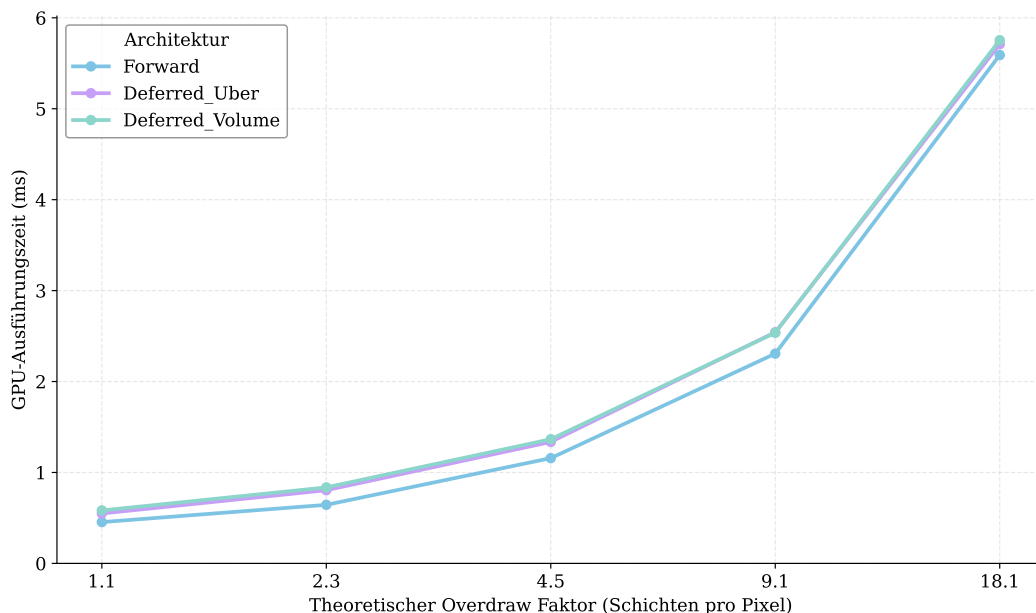


Abbildung 5.3: Messung der GPU-Ausführungszeit in Abhängigkeit vom theoretischen Overdraw-Faktor (1,1 bis 18,1 Schichten pro Pixel) bei einer Skalierung von 2.000 bis 32.000 Instanzen.

Die nahezu deckungsgleichen Kurven demonstrieren, dass die umfangreichen G-Buffer-Schreibvorgänge selbst bei einem äußerst hohen Overdraw keine VRAM-Blockade induzieren. Da opake Geometrie gerendert wird, testet die GPU-Hardware die Tiefe bereits vor dem eigentlichen Fragment-Shader. Es ist zu konstatieren, dass verdeckte Pixel-Fragmente auf Silizium-Ebene verworfen werden, bevor kostspielige Schreiboperationen in den Framebuffer oder G-Buffer stattfinden [12, S. 801].

5.3 Bandbreite und Auflösung

In dem nachfolgenden Kapitel werden Benchmarks und Erkenntnisse präsentiert, die dazu dienen, die bandbreitenspezifischen Limitierungen auszuloten. Im Rahmen der Analyse werden die API und die Speicherkosten des G-Buffers sowie der zunehmende Arithmetic Logic Unit (ALU)-Aufwand des Forward-Rendering evaluiert.

5.3.1 Auflösung

Der folgende Versuch zielt darauf ab, den Einfluss der Framebuffer-Auflösung (und der damit verbundenen Pixel-Anzahl) auf die VRAM-Füllrate und den Shading-Aufwand zu evaluieren. Sowohl die Geometrie als auch die Lichtverhältnisse bleiben auf einen Standard von $N_{\text{Light}} = 100$ Lichtern und $N_{\text{Inst}} = 5.000$ Objekt-Instanzen festgelegt. Im Rahmen der Evaluierung werden ausschließlich die Auflösung und damit gerenderten Pixel analysiert. Da Deferred Rendering die Beleuchtung strikt im 2D-Screen-Space berechnet, wird erwartet, dass die Kosten an die Pixelanzahl gekoppelt sind [20, Abschn. 9.2].

Versuchs-Setup	
Unabhängige Variable:	Basis-Auflösung ($R_{\text{Base}} \in \{480\text{p}, 720\text{p}, 1080\text{p}, 1440\text{p}, 4\text{K}\}$)
Skalierung (Pixel):	409.920 bis 8.294.400
Kontrollvariablen:	$N_{\text{Inst}} = 5.000$, $\text{LOD} = 1$, $N_{\text{Light}} = 200$, Statische Kamera
Zielgrößen:	GPU-Gesamtlaufzeit (t_{GPU})

Bei niedrigen Auflösungen (480p bis 720p) weisen die Architekturen eine relativ geringe Variabilität auf ($t_{\text{GPU}} < 4 \text{ ms}$). Die Forward-Pipeline weist jedoch eine zunehmende Abnahme der Skalierbarkeit auf. Bei 4K werden Forward-Renderzeiten von nahezu 17 ms erreicht, während die Deferred-Pipelines mit etwa 12 bis 13 ms deutlich effizienter sind (siehe Abbildung 5.4).

Die starken Einbußen von Forward gegenüber den Deferred-Pipelines wird im Zusammenhang mit Shading Overdraw vermutet. Da in diesem Setup $N_{\text{Light}} = 200$ Lichter berechnet werden, führt die Forward-Pipeline komplexe Beleuchtungsmathematik auf Fragmenten

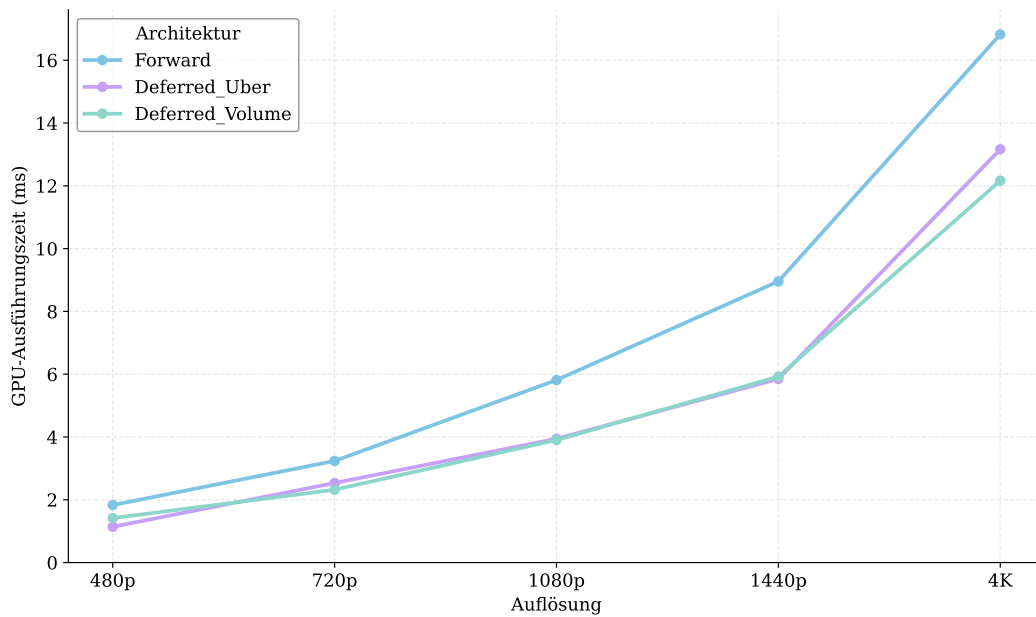


Abbildung 5.4: Messung der GPU-Ausführungszeit in Abhängigkeit von der Framebuffer-Auflösung (480p bis 4K) unter einer statischen Last von 200 Lichtquellen.

aus, die später von anderen Objekten verdeckt werden [7, S. 55]. Bei einer Anzahl von 8,3 Millionen Pixeln (4K) multipliziert sich diese redundante Rechenlast wesentlich stärker als bei den entkoppelten Deferred-Pipelines (ein ALU-Limit wird schneller erreicht). Diese Vervielfachung der redundanten Rechenlast bei steigender Pixelanzahl stützt die Annahme aus Hypothese H1.1, wenngleich das absolute Limit hier durch die ALU-Auslastung und nicht rein durch den geometrischen Overdraw definiert wird.

5.3.2 Bandbreiten

Ziel der folgenden Evaluation ist die Isolierung der absoluten Speicherbandbreite (Fill-Rate) und des Basis-Overheads. Der Versuchsaufbau gründet auf einem simplen binären Test. Die Szene ist in diesem Fall vollständig leer ($N_{\text{Inst}} = 0$, $N_{\text{Light}} = 0$). Es erfolgt lediglich eine Aktivierung bzw. Deaktivierung eines vollflächigen Boden-Quads (in 1080p und 4K Auflösung). Im Rahmen dessen werden die Bandbreiten-Nachteile (Memory Tax) beim Schreiben und Lesen der Framebuffer ausgelotet.

Versuchs-Setup: Base Bandwidth Tax (Dual-Auflösung)

Unabhängige Variablen:	Sichtbarkeit des Bodens ($C_{\text{Floor}} \in \{\text{Deaktiviert}, \text{Aktiviert}\}$), Render-Auflösung ($R \in \{1080p, 4K\}$)
Metrik (Zustände):	0 sichtbare Fragmente vs. 100 % Screen-Space-Abdeckung (≈ 2 Mio. vs. $\approx 8,3$ Mio. Pixel)
Kontrollvariablen:	$N_{\text{Inst}} = 0$, $N_{\text{Light}} = 0$
Zielgröße:	GPU-Gesamtlaufzeit (t_{GPU})

Bei Betrachtung der Abbildung 5.5 lassen sich zwei grundlegende Aspekte feststellen. Die Basislast der Deferred-Pipelines scheint im Vergleich zu der Forward-Pipeline geringfügig höher zu sein. Ansonsten kann keine signifikante prozentuale Steigerung der GPU-Aufwände zwischen den Pipelines in der 1080p Messung festgestellt werden. Bei der Initialisierung des bildschirmfüllenden Boden-Quads zeigen sowohl die Forward- als auch die beiden Deferred-Pipelines eine annähernd identische Skalierung, wobei sich die t_{GPU} -Zeit in allen Fällen um etwa 64 % erhöht. Abbildung 5.6 zeigt jedoch, dass die Deferred-Volume-Pipeline bei einer Auflösung von 4K im Vergleich zu anderen Single-Pass-Architekturen signifikant schlechter skaliert.

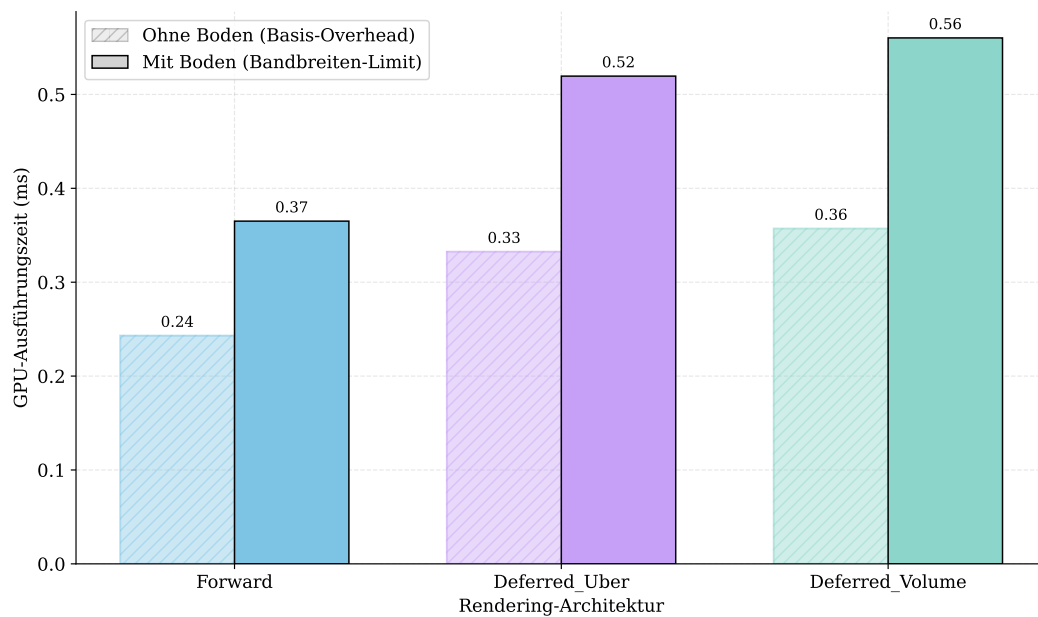


Abbildung 5.5: Vergleich der GPU-Ausführungszeit zwischen keiner Geometrielast (Basis-Overhead) und vollflächiger Screen-Space-Abdeckung bei 1080p Auflösung ohne Beleuchtung.

Die Gegenüberstellung der Messreihen bei 1080p und 4K lässt auf einen physischen Hardware-Engpass schließen. Bei einer Auflösung von 1080p ist die VRAM-Bandbreite noch ausreichend, um den Lese- und Schreibverkehr von circa zwei Millionen Pixeln ohne Verzögerungen abzuwickeln. Bei einer Vervierfachung der Datenlast, wie sie bei 4K-Auflösungen erreicht wird, kommt es jedoch zur Sättigung des Speicherbusses. Wäre die Ursache ein reines Rechenproblem der Shader (ALU-Limit), müssten alle Pipelines bei 4K proportional gleichmäßig skalieren. Die signifikant stärkere Einbuße der Deferred-Volume-Pipeline bei der 4K-Auflösung (240 %) im Vergleich zu den Forward- und Uber-Shader-Ansätzen (135 %) lässt sich primär auf den hardwarenahen Overhead des additiven Blendings (*Read-Modify-Write*) zurückführen und liefert ein starkes Indiz für die Hypothese H3. Deferred-Uber und Forward rechnen das finale Beleuchtungsergebnis in einem einzigen Durchlauf aus (Single-Pass-Architektur) und überschreiben den Zielpuffer direkt (nur eine Replace-Operation). Die Multi-Pass-Architektur der Volume-Pipeline hingegen erfordert zwingend den kontinuierlichen Einsatz von additivem Blending zur Akkumulation der Lichtbeiträge [21, S. 9]. Akenine-

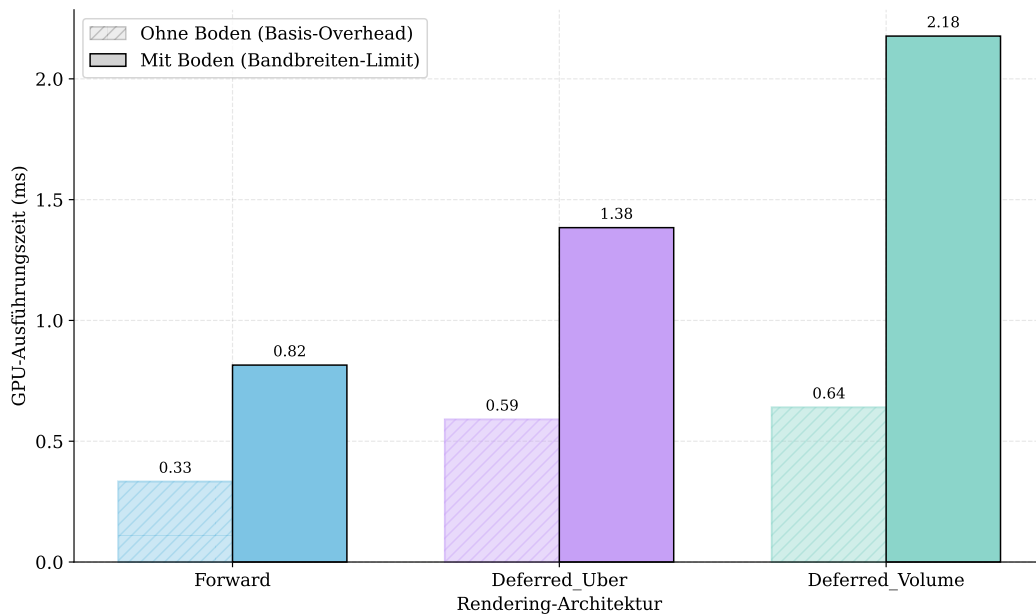


Abbildung 5.6: Vergleich der GPU-Ausführungszeit zwischen keiner Geometrielast (Basis-Overhead) und vollflächiger Screen-Space-Abdeckung bei 4K Auflösung ohne Beleuchtung.

Möller et al. [12, S. 803] weisen darauf hin, dass derartige Blending-Operationen auf dem VRAM-Bus im Vergleich zu einfachen Schreibvorgängen mit signifikant höheren Kosten verbunden sind. In Kombination mit dem simultanen Auslesen der breiten G-Buffer (Multiple Render Targets (MRT)) resultiert dieser zusätzliche Lese-Schreib-Zyklus bei 8,3 Millionen Pixeln (4K) selbst bei simplen Ambient-Auswertungen ($N_{\text{Light}} = 0$) in einer Sättigung der Speicherbandbreite.

Trotz dieser starken theoretischen Indizien muss an dieser Stelle eine methodische Einschränkung des entwickelten Frameworks eingeräumt werden. Da die Testumgebung primär auf die Messung von Ausführungszeiten ausgelegt ist, fehlen native Schnittstellen zur direkten Auslesung spezifischer Hardware-Counter der GPU, wie beispielsweise der expliziten Bus-Auslastung. In dieser Evaluation im Besonderen kann die tatsächliche Ursache für den Leistungsabfall der Deferred-Volume-Pipeline somit nicht zweifelsfrei geklärt werden. Die präzise Evaluierung von Bandbreiten- und Bus-Limitierungen verbleibt daher als eine architektonische Schwäche der vorliegenden Testarchitektur.

5.4 Beleuchtung

Im nachfolgenden Kapitel werden Benchmarks und Erkenntnisse präsentiert, die dazu dienen, die beleuchtungsspezifischen Limitierungen zu ermitteln. Da Deferred Rendering im Bereich der Beleuchtungsberechnung signifikante Vorteile verspricht, sollen im Folgenden

die genauen architektonischen Crossover-Points ermittelt werden, ab denen die Deferred-Pipelines die Forward-Pipelines übertreffen. Auf Hardware-Ebene werden im Rahmen der vorliegenden Untersuchung die ALU- und Raster Operation Pipeline (ROP)-Limits mithilfe der nachfolgenden Benchmarks evaluiert.

5.4.1 Lichtanzahl

Ziel dieser Evaluation ist die Untersuchung der Skalierbarkeit von Punktlichtern. Dazu wird der klassische Ansatz der Uniform Buffer Object (UBO)-Iteration (Forward und Deferred-Uber) mit der Rasterisierung von Proxy-Geometrie (Deferred-Volume) verglichen. Die Anzahl der Lichtquellen (N_{Light}) wird dabei logarithmisch von 10 auf bis zu 5.000 skaliert. Wie in Unterkapitel Kapitel 4.3.1 beschrieben, sind sowohl die Forward-Pipeline als auch die Deferred-Uber-Pipeline hinsichtlich der maximal nutzbaren Lichtquellen durch die UBO-Kapazität auf insgesamt 500 Punktlichter limitiert. Im Rahmen der Untersuchung der allgemeinen Skalierung erfolgt zudem eine Evaluation der Deferred-Volume-Pipeline mit bis zu 5.000 Lichtquellen. Darüber hinaus zeichnet sich das szenische Setup durch eine minimale Überlappung der Lichtquellen und deren Radien aus.

Versuchs-Setup	
Unabhängige Variable:	Anzahl aktiver Lichtquellen ($N_{\text{Light}} \in \{10, \dots, 5.000\}$)
Kontrollvariablen:	Umgebungslicht deaktiviert ($I_{\text{Ambient}} = 0,0$), Licht-Radius reduziert ($r_{\text{Light}} = 1,0$)
Architektonische Limits:	Forward und Deferred-Uber limitiert auf max. 500 Lichter (UBO-Kapazität)
Zielgröße:	GPU-Gesamtlaufzeit (t_{GPU})

Die Ergebnisse, dargestellt in Abbildung 5.7, zeigen eine nahezu identische Verteilung der Pipelines bei einer Lichtanzahl von ≤ 50 Punktlichtern. Bei geringer Last nimmt die Forward-Pipeline eine leicht überlegene Position ein. Bei einer Last von etwa 100 Lichtquellen lässt sich allerdings ein Kipppunkt zugunsten beider Deferred-Pipelines erkennen. Obwohl die Deferred-Uber-Pipeline eine signifikant geringere Skalierbarkeit aufweist als die Deferred-Volume-Pipeline, übertrifft diese ab dem 100-Lichter-Kipppunkt die Forward-Pipeline zunehmend, wenn auch nur marginal. Bei einer Anzahl von 250 Lichtquellen zeigt sich ein signifikanter Einbruch der Leistung von Forward, während Deferred-Volume eine bemerkenswerte Effizienz und flache Skalierung aufweist. Erst bei einer Last von 5.000 Lichtquellen überschreitet die Deferred-Volume-Pipeline die Rechenzeit von Forward bei einer zehnmal geringeren Last von 500 Lichtern.

Aus den Beobachtungen kann abgeleitet werden, dass die Deferred-Pipelines bei steigender Lichtanzahl grundlegend besser skalieren als naive Forward-Ansätze. Diese Erkenntnis validiert die in der Literatur beschriebene Überlegenheit von Deferred-Rendering im Bereich

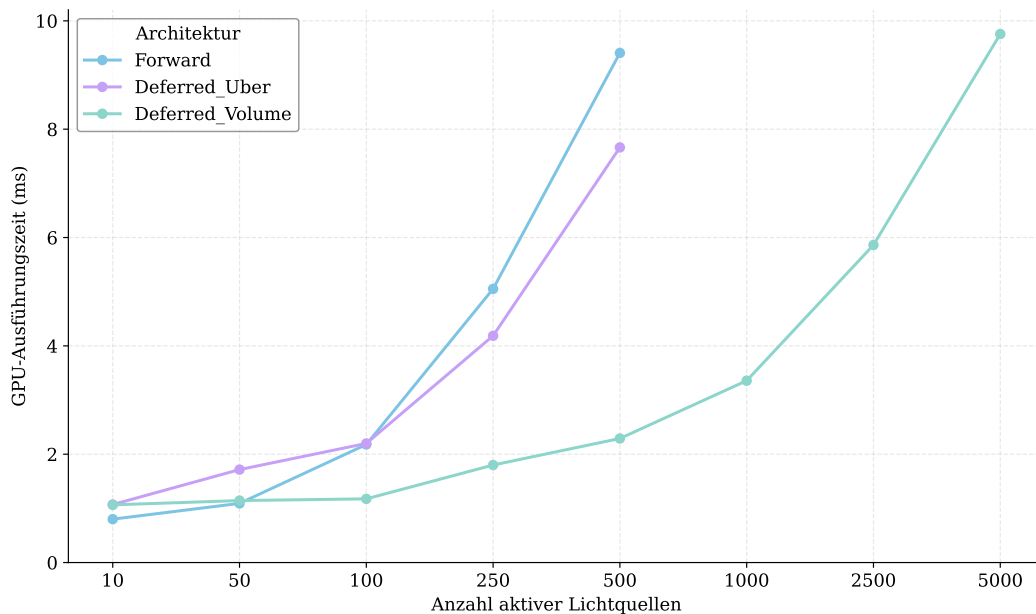


Abbildung 5.7: Messung der GPU-Ausführungszeit in Abhängigkeit von der Anzahl aktiver Lichtquellen (10 bis 5.000).

massiver Lichtevaluationen und bestätigt den primären Existenzgrund dieses Paradigmas im Allgemeinen [8, S. 1]. Obwohl die Deferred-Uber-Pipeline den gleichen UBO-Loop ausführt wie der Forward-Ansatz, wird durch das Entkoppeln der Geometrie der Shading-Overdraw effektiv vermieden. Der Einsatz des G-Buffers gewährleistet, dass die Schleife für die Lichtberechnung lediglich für die auf dem Bildschirm tatsächlich sichtbaren Pixel ausgeführt wird. Die signifikante Überlegenheit des Deferred-Volume-Ansatzes lässt sich auf die Effizienz des lokal begrenzten Evaluationsbereiches der Licht-Volumes zurückführen. Anstelle eines volumenunabhängigen Vollbild-Passes wird seitens der GPU lediglich eine Rasterisierung der Licht-Proxy-Geometrien vorgenommen. Die ALU-Last beschränkt sich somit exakt auf die Screen-Space-Projektion, in der das Lichtvolumen die Geometrie tatsächlich beeinflusst [12, S. 886]. Bei einer extremen Last von 5.000 Lichtquellen lässt sich zudem die Hypothese aufstellen, dass das Limit der Deferred-Volume-Pipeline primär durch die VRAM-Bandbreite der ROPs aufgrund des massiven *Read-Modify-Write*-Zyklus des additiven Blendings erreicht wird (siehe Kapitel 5.3.2). Diese Beobachtung verifiziert den algorithmischen Kern von Hypothese H1.1: Die multiplikative Komplexität der direkten Beleuchtungsberechnung zwingt die Forward-Architektur bei steigender Lichtdichte unweigerlich in ein massives Performance-Defizit.

5.4.2 Lichtintensität & Overlap

Ziel der folgenden Untersuchungen ist die Evaluation der Pipelines unter Bedingungen extremer räumlicher Dichte der Lichtquellen und des damit verbundenen massiven *Light Overdraws*. Es werden zwei komplementäre Szenarien evaluiert: Test A skaliert den physi-

kalischen Radius von 250 Lichtquellen, um die Bildschirmabdeckung künstlich in die Höhe zu treiben, während Test B bis zu 1.000 Lichtquellen auf exakt dieselbe 3D-Koordinate (Singularität) zwingt, um eine maximale Überlagerung pro Pixel zu garantieren. Dieses Versuchsdesign verspricht die gezielte Provokation und Isolierung des *Read-Modify-Write*-Bandbreitenlimits der Rasterisierungseinheiten. Es soll aufgezeigt werden, an welchem Punkt die kontinuierliche Speicher-Akkumulation bei überlappenden Lichtvolumina die reine ALU-Rechenlast der Shader-Schleifen als primärer Performance-Flaschenhals ablöst.

Versuchs-Setup

Test A: Skalierung der Lichtintensität (Radius)

Unabhängige Variable: Licht-Radius-Multiplikator ($r_{\text{Light}} \in \{0,5, \dots, 20,0\}$)

Kontrollvariablen: $N_{\text{Light}} = 250$ (fixiert), Umgebungslicht deaktiviert ($I_{\text{Ambient}} = 0,0$)

Test B: Licht-Singularität (Räumliche Ballung)

Unabhängige Variable: Anzahl aktiver Lichtquellen ($N_{\text{Light}} \in \{10, \dots, 1.000\}$)

Kontrollvariablen: Licht-Singularität aktiviert ($S_{\text{Light}} = \text{Aktiviert}$), $I_{\text{Ambient}} = 0,0$

Zielgröße: GPU-Gesamtlaufzeit (t_{GPU})

Die Analyse der Abbildung 5.8 zeigt, dass die Deferred-Volume-Pipeline bei einem kleinen Radius von 0,5 und dem damit verbundenen minimalen Volume-Overlap den signifikant schnellsten Ansatz darstellt. Erhöht man den Radius-Multiplikator auf 20,0 und steigert damit die Screen-Space-Abdeckung drastisch, so führt dies zu einer massiven Zunahme der Laufzeit der Deferred-Volume-Pipeline auf über 8 ms, während die Werte für Forward und Deferred-Uber nahezu konstant bleiben. In Abbildung 5.9 wird ein identisches Bild in forcierter Form dargestellt. Die Deferred-Volume-Pipeline bricht bei einer Skalierung auf bis zu 1.000 Lichtquellen stark ein und erreicht eine GPU-Laufzeit von nahezu 37 ms. Die Deferred-Uber-Pipeline und der Forward-Ansatz zeigen demgegenüber bei einer Auslastung von bis zu 500 Lichtquellen eine signifikant höhere Effizienz und verbleiben stabil unterhalb der 12 ms-Marke.

Diese Untersuchungen forcieren die in den Kapiteln 5.3.2 und 5.4.1 beobachteten Phänomene im Zusammenhang mit den ROP- und Bandbreitenlimits der Deferred-Volume-Pipeline und verifizieren Hypothese H3 abschließend: Sie verorten die größte architektonische Limitierung des Deferred-Volume-Ansatzes in Szenarien mit hochgradig überlappenden Einflussbereichen verschiedener Lichtquellen. Wenn hunderte Licht-Volumina dieselbe Pixelfläche abdecken, zwingt die Deferred-Volume-Pipeline die GPU, für diese einzelnen Pixel hunderte Male den RMW-Zyklus für das additive Blending (ROPs) auf dem Speicherbus auszuführen [21, Folie 9], [19, S. 676]. Der VRAM-Speicherbus kollabiert in der Folge unter der schieren Frequenz dieser redundanten Lese- und Schreibzugriffe.

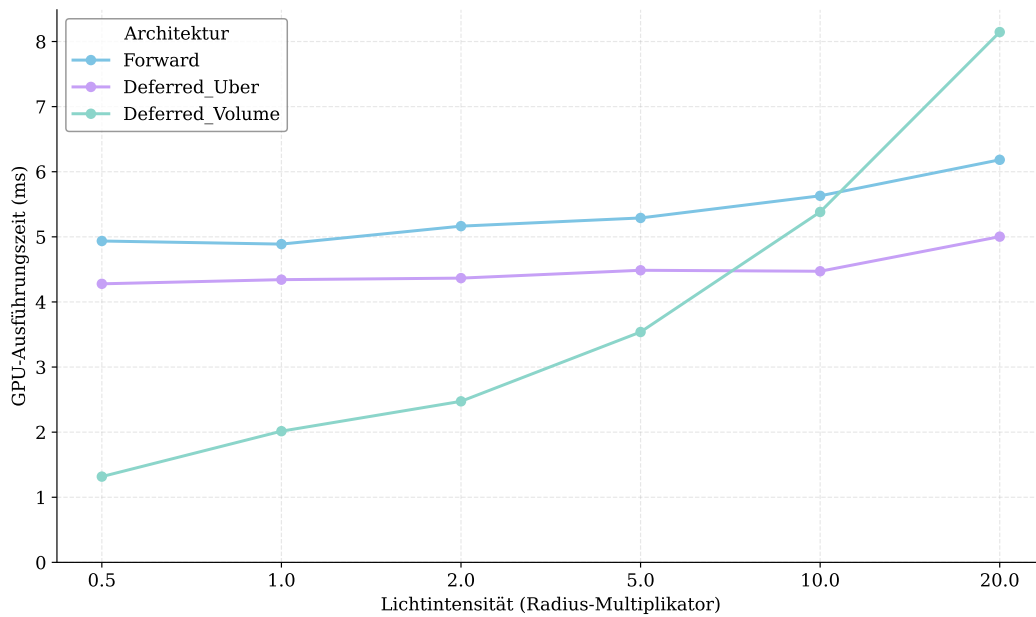


Abbildung 5.8: Messung der GPU-Ausführungszeit in Abhängigkeit von der Lichtintensität respektive dem Radius-Multiplikator (0,5 bis 20,0) bei einer konstanten Anzahl von 250 Lichtquellen.

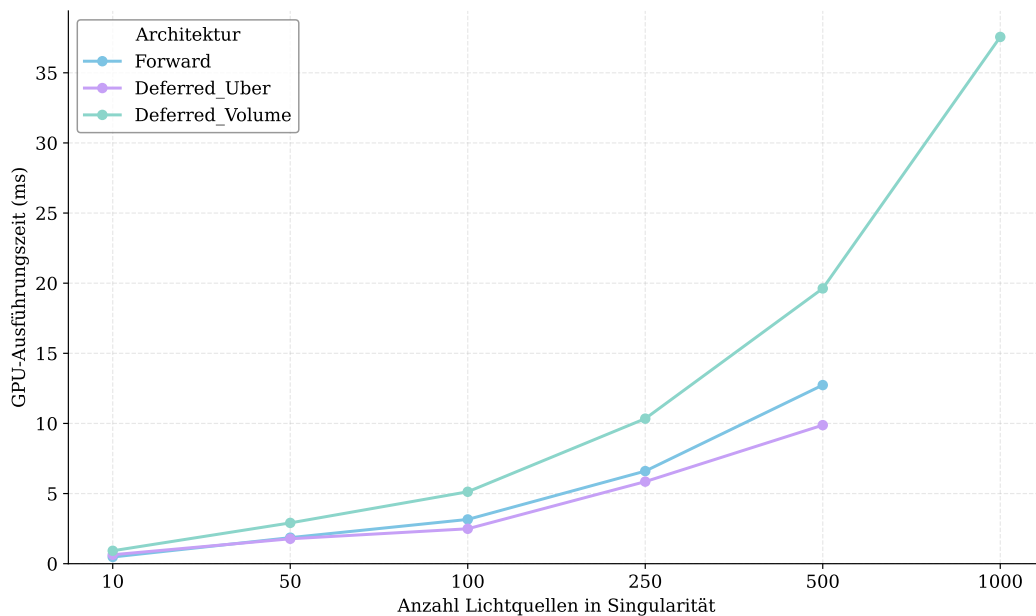


Abbildung 5.9: Messung der GPU-Ausführungszeit in Abhängigkeit von der Anzahl der Lichtquellen (10 bis 1000) bei einer Positionierung aller Lichtquellen in einer räumlichen Singularität.

5.4.3 Shading Overdraw

Wie in Unterkapitel 5.2.3 bereits für die reine Geometrie evaluiert, wird im Folgenden erneut der Einfluss des Fragment-Overdraws untersucht. In diesem Versuchsaufbau wird die Tiefenkomplexität allerdings unter Zuhilfenahme einer konstanten Beleuchtungslast überprüft. Der Aufbau ist hierbei nahezu identisch zu Unterkapitel 5.2.3, mit der Ausnahme von 100 zusätzlich aktiven Lichtquellen. Die Anzahl der Lichtquellen ist dabei so gewählt, dass sie den in Kapitel 5.4.2 aufgezeigten *Light-Volume-Overlap* innerhalb der Deferred-Volume-Pipeline möglichst geringfügig provoziert. Für die Messergebnisse wird erwartet, dass der Forward-Ansatz durch klassischen Shading-Overdraw an Effizienz verliert, da die aufwendige Beleuchtungsberechnung hier untrennbar mit der Geometrieauswertung verknüpft ist. Somit sollte sich der Berechnungsaufwand theoretisch mit jedem überzeichneten Fragment vervielfachen.

Versuchs-Setup	
Unabhängige Variable:	Instanzanzahl ($N_{\text{Inst}} \in \{2.000, \dots, 32.000\}$)
Kontrollvariablen:	$N_{\text{Light}} = 100$, Spawn-Radius reduziert ($r_{\text{Spawn}} = 30,0$), Boden deaktiviert ($C_{\text{Floor}} = \text{Deaktiviert}$)
Metrik (X-Achse):	Theoretischer Overdraw-Faktor ($\approx 1,1$ bis $18,1$)
Zielgröße:	GPU-Laufzeit (t_{GPU}), aufgeschlüsselt nach Komponenten

Abbildung 5.10 zeigt, dass der vorgelagerte G-Buffer-Pass der Deferred-Pipelines in diesem Kontext den primären weitestgehend linearen Skalierungsfaktor bei einer ansteigenden Instanzdichte für die beiden Deferred-Pipelines darstellt. Bei der Betrachtung der Forward-Pipeline-Renderzeit ist ein Anstieg der kombinierten Zeit zu verzeichnen (von 1.11 ms auf 5.98 ms, der jedoch nicht so katastrophal ausfällt, wie es die ursprüngliche Annahme vermuten ließe. Der Engine Overhead weist zudem bei sämtlichen Pipelines eine hohe Konstanz auf.

Die Ergebnisse dieser Evaluation widerlegen die weit verbreitete Annahme, dass der Fragment Overdraw, insbesondere in Kombination mit Beleuchtung, in naiven Forward-Pipelines einen der größten Engpässe darstellt [8, Abschn. 1]. Die Ergebnisse dieser Evaluation erzwingen somit eine Relativierung der Hypothese H1.1: Durch hardwarebasierte Optimierungen wie *Early Z* wird auf modernen Grafikkarten auch ohne dedizierten, softwarebasierten *Z-Prepass* ein signifikanter Teil der redundanten Rechenlast der Beleuchtungsberechnung verdeckter Fragmente vermieden [12, S. 801 u. 803].

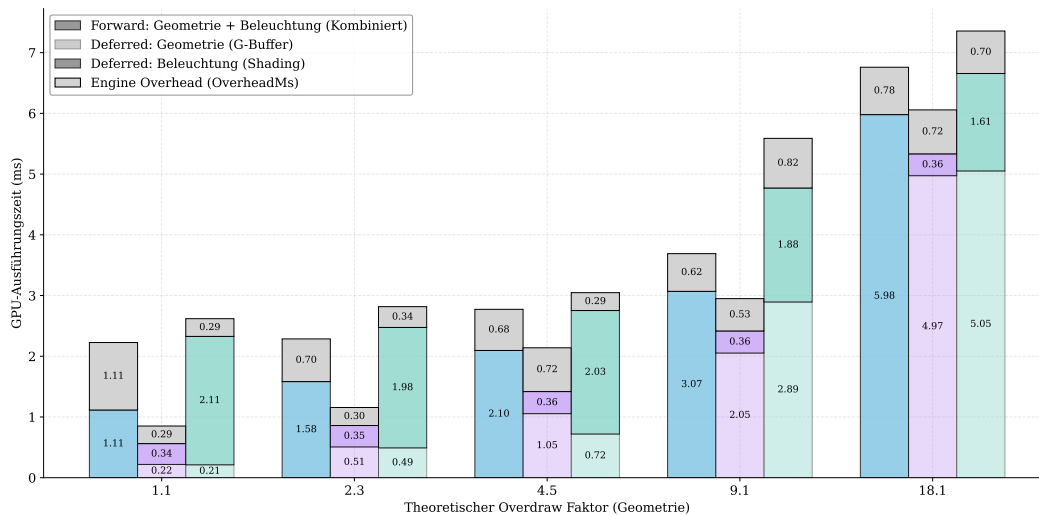


Abbildung 5.10: Aufschlüsselung der GPU-Ausführungszeit nach einzelnen Pipeline-Komponenten in Abhängigkeit vom theoretischen Overdraw-Faktor bei einer konstanten Last von 100 Lichtquellen.

5.5 Hybride Stilisierung & Entropie

In den vorangegangenen Kapiteln 5.2 bis 5.4 wurde das grundlegende Verhalten der evaluierten Rendering-Architekturen aufgezeigt und somit zugleich die Fähigkeiten und Grenzen des HyDra-Frameworks demonstriert. Die nachfolgenden Tests widmen sich dem Kern der Forschung und evaluieren die Wechselwirkungen von Stil-Entropie und den allgemeinen Einfluss selektiver NPR-Techniken auf die verschiedenen Pipelines. Der Fokus liegt hierbei, insbesondere auf der Evaluation von Warp Divergence und Register Pressure.

5.5.1 Stil-Entropie

In den nachfolgenden, zusammengefassten Versuchen werden die Einflüsse von stilistischer Komplexität mittels der Anwendung diverser NPR-Stile analysiert. Neben dem bisher standardmäßig verwendeten Blinn-Phong-Shading finden auch Toon-Shading und Gooch-Shading Anwendung. In der vorliegenden Untersuchung A wird eine geklusterte (niedrige Entropie) mit einer gestreuten (hohe Entropie) Objektverteilung verglichen, wobei die Anzahl der Stile konstant bei drei bleibt. In der weiteren Untersuchung B wird die Anzahl der simultan aktiven Shading-Stile von 1 bis 3 (Blinn, Gooch, Toon) skaliert. In diesem verteilten Szenario (Versuch B) führt die schrittweise Zunahme der Stile zudem zur Isolation des algorithmischen Einflusses der NPR-Effekte. Die vorliegende Untersuchung zielt darauf ab, die Hypothesen der Warp-Divergenz und des Registerdrucks zu analysieren und zu verifizieren. Diese Analyse erfolgt insbesondere mit Blick auf die Performance der Deferred-Über-Pipeline.

Versuchs-Setup

Test A: Räumliche Entropie

Unabhängige Variable: Räumliche Verteilung ($E_{\text{Spatial}} \in \{\text{Geklustert, Gestreut}\}$)

Kontrollvariablen: $N_{\text{Inst}} = 15.000$, $N_{\text{Light}} = 250$, $r_{\text{Spawn}} = 175,0$, Drei aktive Stile ($S_{\text{Count}} = 3$)

Test B: Kombinatorische Degradation

Unabhängige Variable: Anzahl aktiver Rendering-Stile ($S_{\text{Count}} \in \{1, 2, 3\}$)

Kontrollvariablen: Gleicher Aufbau wie Test A, Räumliche Entropie geklustert ($E_{\text{Spatial}} = \text{Geklustert}$)

Zielgröße: GPU-Beleuchtungs-Zeit (t_{Light})

In beiden Ergebnisgraphen (siehe Abbildung 5.11 & 5.13) ist zu erkennen, dass sowohl Forward als auch Deferred-Volume bei gestreuter Verteilung der Stile (hohe Entropie) im Vergleich zu sortierten Verhältnissen extrem stabil bleiben. Auch der erhöhte algorithmische Aufwand in der Lichtberechnung, der durch die komplexeren Aufwände der Toon- und Gooch-Shader entsteht, zeigt nur minimalen Einfluss auf Forward- und Deferred-Volume. Die Deferred-Uber-Pipeline weist in beiden Szenarien allerdings einen signifikanten Einbruch auf und verdoppelt nahezu (Anstieg der Beleuchtungsberechnungs-Zeiten um $\approx 85\%$) ihren Aufwand bei einer gestreuten Stilverteilung (hohe Entropie). Abbildung 5.12 zeigt den Versuchsaufbau innerhalb des *HyDra* Frameworks.

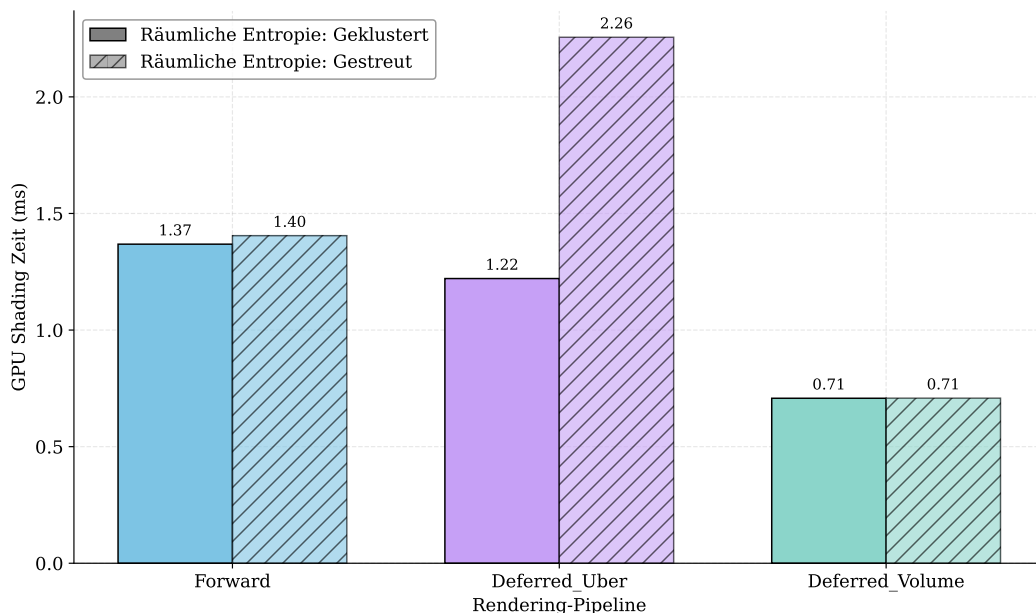
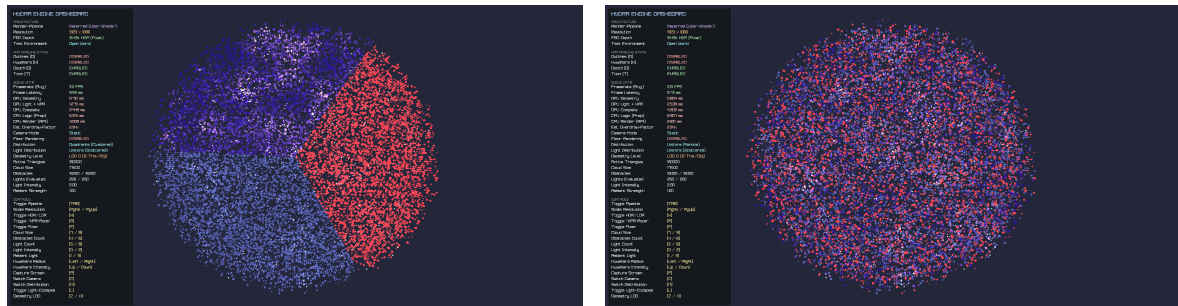


Abbildung 5.11: Messung der GPU-Shading-Zeit in Abhängigkeit von der räumlichen Verteilung (geklustert vs. gestreut) der Rendering-Stile.

In den theoretischen Grundlagen (vgl. Kapitel 2.2.4) wurde dargelegt, dass die GPU Instruktionen im Single Instruction, Multiple Threads (SIMT)-Verfahren in sogenannten *Warps* verarbeitet. Der signifikante Leistungseinbruch der Deferred-Uber-Pipeline im gestreuten Szenario stellt die praktische Manifestation der *Warp Divergence* dar und verifiziert



(a) Geklusterte Verteilung

(b) Gestreute Verteilung

Abbildung 5.12: Vergleich der Stil-Entropie Verteilungen in *HyDra* während der 5.5.1 Test-Suite in Test A (Räumliche Entropie).

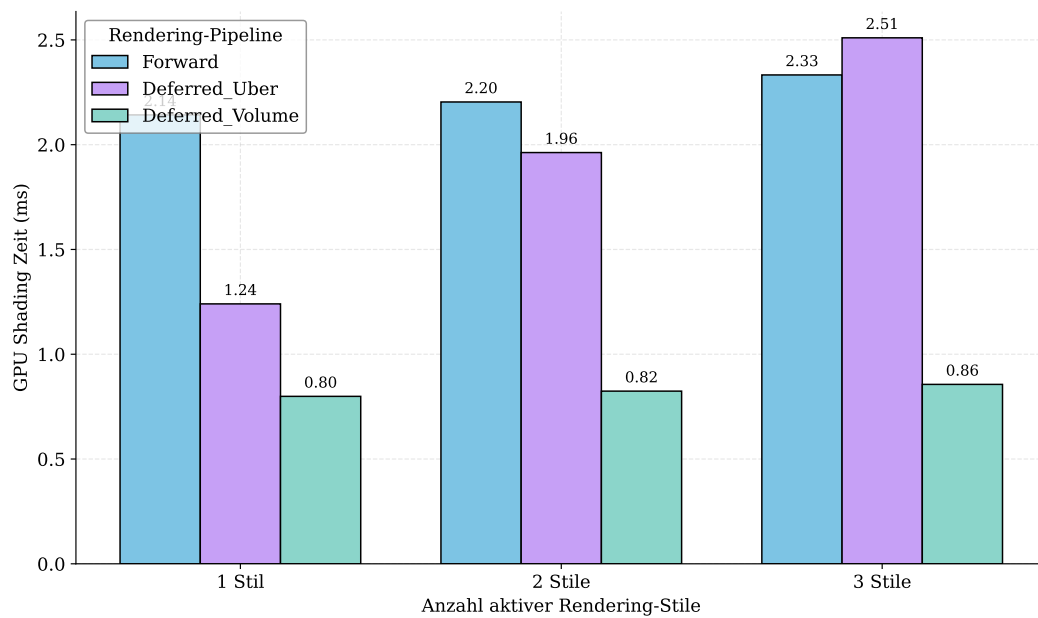


Abbildung 5.13: Messung der GPU-Shading-Zeit in Abhängigkeit von der Anzahl simultan aktiver Rendering-Stile (1 bis 3 Stile).

Hypothese H2 vollumfänglich. Da benachbarte Pixel innerhalb eines Warps unterschiedliche Shading-Modelle evaluieren, wird die Hardware durch diese räumliche Entropie dazu gezwungen, die divergierenden Code-Pfade sequenziell abzuarbeiten. Die Forward- und Deferred-Volume-Architekturen hingegen erweisen sich als systembedingt immun gegen diesen architektonischen Engpass. Die strikte Trennung der *Draw-Calls* nach Material (Forward) oder die Nutzung von mehreren insgesamt kleineren Shader-Pässen (Deferred-Volume) gewährleistet eine Homogenität der aktiven Warps [12, S. 886].

In Bezug auf die lineare Degradation des Deferred-Shaders bei der Kombination mehrerer Stile lässt sich die zusätzliche Hypothese eines erhöhten Register-Drucks aufstellen. Die Implementierung eines zusätzlichen Rendering-Stils im monolithischen Code-Pfad bedingt potenziell die Allokation weiterer Vector General Purpose Register / Vektorregister (VGPR) pro Thread. Es kann deduziert werden, dass sich dadurch die globale Auslastung (Occupancy) der Recheneinheiten verringert (vgl. Kapitel 2.2.3). An dieser Stelle ist jedoch eine methodische Limitierung der vorliegenden Testarchitektur zu berücksichtigen. Aufgrund der Tatsache, dass die Messung der isolierten Beleuchtungs-Laufzeit (t_{Light}) keine Low-Level-Telemetriedaten liefert, kann diese spezifische Ursache nicht abschließend empirisch verifiziert werden. Ohne den Einsatz dedizierter Hardware-Profiler lässt sich nicht zweifelsfrei differenzieren, inwiefern ein erhöhter Registerdruck zum Leistungsabfall der Deferred-Uber-Pipeline beiträgt.

5.5.2 Kernel Bandbreiten Limits

Der folgende Versuch fokussiert sich ausschließlich auf die Deferred-Pipelines (Deferred-Uber und `textitDeferred_Volume`). Die vorliegende Untersuchung widmet sich der Analyse der Einflüsse kernelbasierter NPR-Effekte. Aufgrund der Tatsache, dass solche kernelbasierten Effekte in der Forward-Pipeline auf Objektebene nicht realistisch anwendbar sind (vgl. Kapitel 4.3.1) und nur auf gesamter Screen-Space-Ebene anwendbar sind, kann keine Zielbild-Parität erreicht werden. Ein aussagekräftiger Vergleich mit den Deferred-Pipelines ist somit nicht möglich. Für die nachfolgende Evaluation wird der implementierte Kuwahara-Filter herangezogen. Der Suchradius (R) des stilisierenden Kuwahara-Filters wird sukzessive von 2 auf bis zu 16 Pixel erweitert. Da der Filter im zweidimensionalen Raum agiert, wächst die Anzahl der Textur-Samples quadratisch ($O(R^2)$).

Versuchs-Setup

Unabhängige Variable:	Kuwahara-Kernel-Radius ($R \in \{2, 4, 8, 12, 16\}$)
Kontrollvariablen:	Alle Objekte nutzen Kuwahara, $N_{\text{Inst}} = 15.000$, $N_{\text{Light}} = 250$, Räumliche Entropie gestreut ($E_{\text{Spatial}} = \text{Gestreut}$)
Architektur-Limitierung:	Forward-Pipeline übersprungen (keine native Kuwahara-Unterstützung)
Zielgröße:	Beleuchtungs-Laufzeit (t_{Light})

Wie in Abbildung 5.14 zu erkennen ist, verhalten sich Deferred-Uber und Deferred-Volume bei kleinen Radien ($R = 2$ bis $R = 4$) mit einer Zeit von 1.8 ms bis 3.0 ms nahezu identisch. Ab einem Kuwahara-Radius von 8 (Probe der umliegenden Pixel) lässt sich allerdings ein langsamer Trend zugunsten der Deferred-Uber-Pipeline erkennen. Bei einem Radius von 16 ist Deferred-Uber um fast ≈ 2.5 ms schneller als Deferred-Volume bezüglich der Beleuchtungs-Laufzeit. Die folgende Darstellung der steigenden Komplexität bei Erhöhung des Kuwahara-Radius dient der Vollständigkeit sowie der kontextuellen Einordnung der Graphen und der Effizienz der Rendering-Pipelines: Da die Anzahl der Texturzugriffe T im zweidimensionalen Filterraum quadratisch mit dem Radius R skaliert, lässt sich der theoretische Komplexitätszuwachs über das Verhältnis der Kernel-Flächen bestimmen. Es ergibt sich beim Sprung von $R = 2$ (27 Samples) auf $R = 16$ (1.175 Samples) der exakte Skalierungsfaktor:

$$F = \frac{T(16)}{T(2)} \approx 43,5$$

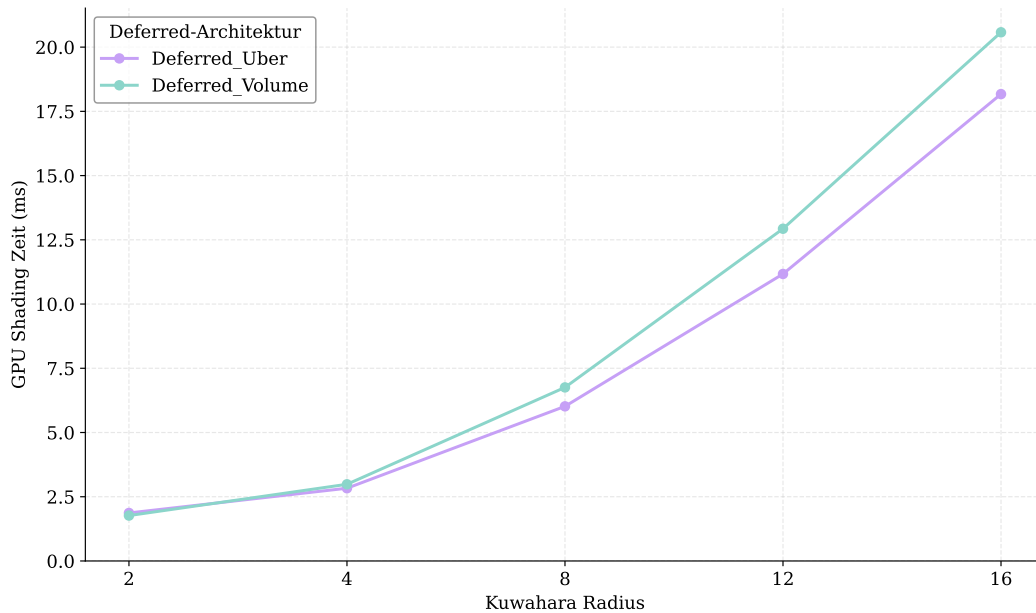


Abbildung 5.14: Messung der GPU-Shading-Zeit in Abhängigkeit vom Kuwahara-Kernel-Radius (2 bis 16).

Die Tatsache, dass die Deferred-Volume-Pipeline im Vergleich zur Deferred-Uber-Pipeline eine geringere Effizienz aufweist, lässt sich auf eine algorithmische Asymmetrie in der Filter-Implementierung zurückführen. Wie bereits in Kapitel 4.2.2 dargelegt, erfordern die beiden Architekturen unterschiedliche Ansätze bei der Evaluierung des Kuwahara-Filters. Die Deferred-Uber-Pipeline operiert als *Pre-Lit*-Filter, um einen algorithmischen Ineffizienz-Kollaps durch redundante Beleuchtungsberechnungen pro Nachbarpixel zu verhindern. Sie liest innerhalb der $O(R^2)$ -Schleife ausschließlich Basis-Materialdaten (Albedo und Style-IDs) aus und berechnet die aufwendige Beleuchtung erst nach Abschluss des Filters für das finale, gemittelte Fragment. Die Deferred-Volume-Architektur appliziert den Filter im *Resolve*-Pass hingegen auf das bereits akkumulierte Licht der Szene (*Post-Lit*).

Dies ermöglicht zwar eine „echtere“ Darstellung des Kuwahara-Effektes, erzwingt jedoch innerhalb der innersten Kuwahara-Schleife einen zusätzlichen Texturlesezugriff auf den *Light Accumulation Buffer* (siehe algorithmische Beschreibung in Kapitel 4.2.2).

Über die Auswirkungen der asymmetrischen Implementationen des Kernel-Filters (Kuwahara) hinaus kann kein signifikanter Unterschied in der Handhabung von Kernel-Effekten zwischen den untersuchten Deferred-Pipelines festgestellt werden.

5.6 Macro-Benchmark Synthese

In der finalen Testreihe erfolgt die Integration aller übergeordneter Einflüsse aus den vorangegangenen Benchmarks in ein umfassendes Real-World-Szenario, um die Interaktion der bislang untersuchten Effekte in ihrer Gesamtheit zu analysieren. Die nachfolgenden Evaluationen werden in einem Schritt zusammengefasst, bestehen allerdings aus drei verschiedenen Einzelpässen, in welchen die verschiedenen Einflüsse von Geometrie, Beleuchtung und selektiver Silisierung gemeinsam betrachtet werden.

Die Makro-Benchmark-Synthese nutzt einen deterministischen Kamera-Flythrough, um die Leistungsfähigkeit der Architekturen in einem praxisnahen Produktionsszenario zu evaluieren. Das zugrunde liegende Szenario generiert eine dichte, prozedurale Umgebung mit 15.000 verteilten Instanzen bei einem konstanten Detailgrad ($LOD = 1$) und vollflächiger Bodenabdeckung. Dabei ist die räumliche Entropie der Rendering-Stile bewusst auf eine gestreute Verteilung konfiguriert. Der programmierte Kamera-Flug erstreckt sich über 3.000 Frames und durchquert die Szenerie auf einem festgelegten Pfad. Dabei werden kontinuierlich wechselnde Sichtfeldkomplexitäten, stark schwankende Frustum-Culling-Raten sowie variable Tiefen- und Lichtüberlappungen erzwungen, um eine ganzheitliche und dynamische Belastung der Pipelines zu gewährleisten. Die Evaluation erfolgt in drei isolierten, aufeinander aufbauenden Pässen zur exakten Dekonstruktion der Leistungskosten. Die Ergebnisse werden kombiniert dargestellt. In Pass 1 (*Geometry Baseline*) wird vollständig auf Beleuchtung und Stilisierung verzichtet, um die reine Grundlast der Geometrieverarbeitung und G-Buffer-Initialisierung zu erfassen. In Pass 2 (*Shading Tax*) werden 250 dynamische Punktlichter mit aktivierter Umgebungsbeleuchtung hinzugefügt, um die Effizienz der additiven Lichtakkumulation zu messen. In Pass 3 (*NPR-Parity*) werden zusätzlich die hybriden Stilisierungen (Toon-Shading, Gooch-Shading und Outlines) aktiviert, um die kombinierte Auswirkung von Beleuchtungs- und Entropielast zu evaluieren. Als zentrale Metrik wird in allen Messungen die akkumulierte GPU-Gesamtlaufzeit (t_{GPU}) herangezogen.

Versuchs-Setup

Versuchs-Typ:	Dynamischer Kamera-Flythrough (3.000 Frames Gesamtdauer)
Kontrollvariablen:	$N_{\text{Inst}} = 15.000$, $r_{\text{spawn}} = 150,0$, $LOD = 1$, Boden aktiviert, Gestreute Stile ($E_{\text{Spatial}} = \text{Gestreut}$)
Pass 1 (Geometry Baseline):	$N_{\text{Light}} = 0$, Alle NPR-Effekte deaktiviert
Pass 2 (Shading Tax):	$N_{\text{Light}} = 250$, Alle NPR-Effekte deaktiviert
Pass 3 (NPR-Parity):	$N_{\text{Light}} = 250$, Gooch, Toon und Outlines aktiviert
Zielgröße:	GPU-Gesamtlaufzeit (t_{GPU})

Die Betrachtung der akkumulierten Einzelpässe in Abbildung 5.15 schlüsselt die architektonischen Kostenstrukturen der Rendering-Pipelines detailliert auf. Im ersten Pass (Geometry Baseline) erweist sich die Forward-Architektur mit einer initialen Ausführungszeit von 0.51 ms als minimal effizienter als die Deferred-Architekturen. Die Deferred-Pipelines weisen mit 0.68 ms (Deferred-Uber) respektive 0.77 ms (Deferred-Volume) die erwartete architektonische Grundlast auf, die durch die initialen Schreibvorgänge in den G-Buffer bedingt ist (vgl. Basislast in 5.3.2). Unter der additiven Zuschaltung von 250 Lichtquellen im zweiten Pass (Shading Tax) kollabiert die Forward-Pipeline unter der multiplikativen Instruktionslast der Lichtberechnungen und verzeichnet eine massive Zusatzlast von +1.69 ms. Die Deferred-Pipelines profitieren hier signifikant von der zweidimensionalen Screen-Space-Beleuchtung und fangen die Rechenlast mit 0.72 ms (Deferred-Uber) sowie +1.01 ms (Deferred-Volume) deutlich effizienter ab (vgl. Ergebnisse aus 5.4.1). Die Aktivierung der gestreuten NPR-Stile im dritten Pass (NPR-Parity) offenbart schließlich die architektonischen Schwächen des Deferred-Uber-Shaders, der durch *Warp Divergence* und Registerdruck eine Zusatzlast von 0.10 ms erfährt (vgl. 5.5.1). Das Deferred-Volume-Rendering erweist sich mit einer minimalen Erhöhung von lediglich +0.03 ms als nahezu immun gegen diese stilistische Entropie, während die Forward-Pipeline um weitere +0.17 ms degradiert. Der zeitliche Verlauf des Kamera-Flythroughs in Abbildung 5.16 bestätigt diese Beobachtungen über einen Zeitraum von 3.000 Frames. Es lässt sich deutlich erkennen, dass die Leistungskurven mit zunehmender Komplexität des Sichtfelds ansteigen. Sobald die Kamera in das Objekt- und Licht-Cluster eintaucht, können die jeweiligen Spitzen beobachtet werden.

Die Ergebnisse dieser Makro-Synthese vereinen die in den vorangegangenen Evaluierungskapiteln isoliert gewonnenen Erkenntnisse in einem kohärenten Gesamtbild. Als primäres Vorab-Fazit lässt sich konstatieren, dass es im Kontext der untersuchten Paradigmen keine universell überlegene Pipeline-Architektur gibt. Die Leistungsfähigkeit der einzelnen Architekturen unterliegt einer dynamischen Veränderung, die in direkter Abhängigkeit zur dominanten Szenenlast steht. Die naive Forward-Pipeline zeigt eine hohe Effizienz bei der reinen Geometrieverarbeitung und einer geringen Lichtdichte (vgl. Ergebnisse aus 5.2). Aufgrund

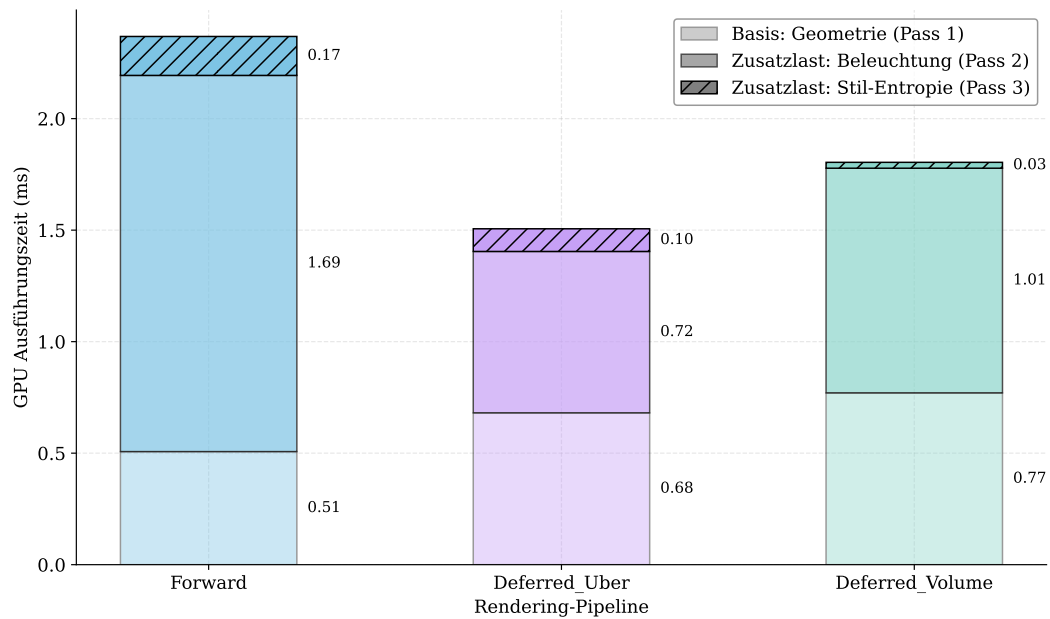


Abbildung 5.15: Aufschlüsselung der GPU-Ausführungszeit nach Pipeline-Komponenten (Geometrie, Beleuchtung und Stil-Entropie).

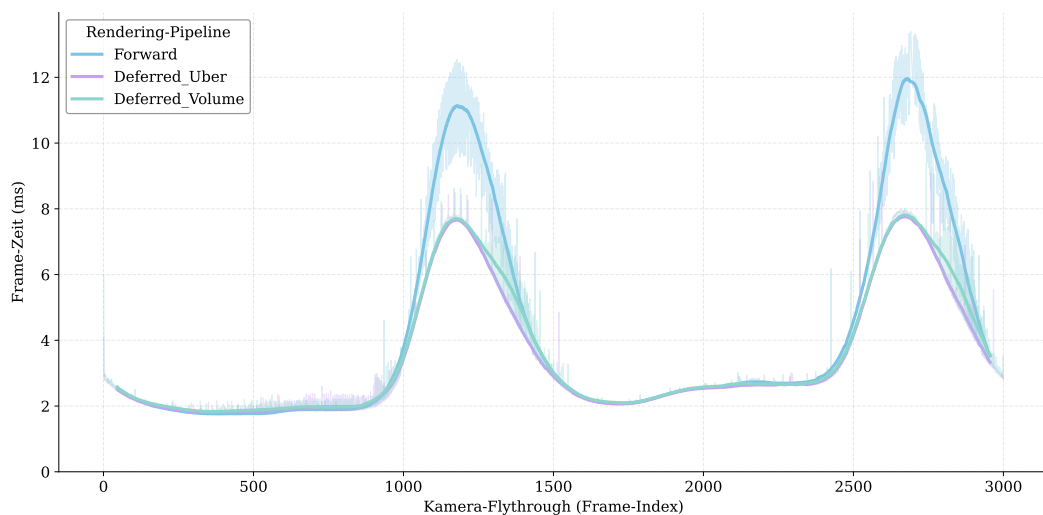


Abbildung 5.16: Verlauf der Frame-Zeit über die Dauer von 3000 Frames des Kamera-Flythroughs unter kombinierter Geometrie-, Beleuchtungs- und Stil-Last.

der untrennbaren Verknüpfung von Geometrieauswertung und Beleuchtungsberechnung kollabiert sie jedoch unter der multiplikativen Instruktionslast, sobald lichtstarke Produktionsszenarien mit hunderten Lichtquellen gefordert werden (vgl. Ergebnisse aus 5.4.1). Die einfache Deferred-Uber-Architektur löst dieses Beleuchtungsproblem hocheffizient im Screen-Space, scheitert jedoch architektonisch an der *Warp Divergence*, sobald eine hohe stilistische Entropie durch divergierende NPR-Code-Zweige erzwungen wird (vgl. Ergebnisse aus 5.5.1). Die Deferred-Volume-Architektur umgeht diese Branching-Problematik zwar durch die strikte architektonische Isolierung der Code-Pfade in Proxy-Geometrien, weist jedoch fundamentale Schwächen in der Speicherbandbreite auf. Ihr primärer architektonischer Engpass manifestiert sich auf dem VRAM-Speicherbus, der durch zahlreiche additive RMW-Zyklen bei extremer räumlicher Überlappung der Lichtquellen gesättigt wird (vgl. Ergebnisse aus 5.4.2).

6 Diskussion

Dieses Kapitel führt die theoretischen Anforderungen und die praktischen Messergebnisse zusammen. Der Fokus liegt auf der Bewertung der Daten, der Vorstellung der heuristischen Entscheidungsmatrix als Kernbeitrag der Arbeit sowie einer kritischen Betrachtung der eigenen Arbeitsweise und der Erfahrungen bei der Entwicklung.

6.1 Zusammenführung und Interpretation der Testergebnisse

Abgleich der gemessenen Kurven mit den zuvor aufgestellten Vermutungen aus den Tests. Beschreibung der genauen Belastungsgrenzen aller drei Render-Wege bei steigender Last durch Lichter, Geometriedichte und durcheinandergewürfelte Zeichenstile im Bild.

6.2 Die heuristische Entscheidungsmatrix

Vorstellung des eigentlichen Ergebnisses der Arbeit in Form einer übersichtlichen Entscheidungsmatrix für Entwickler. Bereitstellung von konkreten Empfehlungen für den Aufbau einer Grafik-Engine basierend auf Geometriedichte, Lichtanzahl und gewünschten Zeichenstilen, wobei die Matrix in der praktischen Anwendung wie ein Kochbuch Schritt für Schritt durch die Architekturentscheidung führt.

6.3 Kritische Reflexion und Limitationen

Ehrliche Betrachtung der Schwachstellen und Grenzen des gewählten Versuchsaufbaus. Diskussion über bewusst weggelassene Faktoren wie andere Lichttypen, transparente Objekte oder die Ausführung auf mobilen Geräten und deren Auswirkung auf die Aussagekraft der Messergebnisse.

6.4 Lessons Learned

Betrachtung von Nutzen und Aufwand aus Sicht der Softwareentwicklung abseits der reinen Performance-Messwerte. Vergleich zwischen der schwierigen Wartung von sehr großen Shader-Dateien und der besseren Modularität anderer Ansätze sowie Probleme beim fehlerfreien Einbau der Zeitmessung.

7 Zusammenfassung und Ausblick

Dieses abschließende Kapitel fasst die gesamte Arbeit kurz zusammen und gibt einen Ausblick auf zukünftige Entwicklungen.

7.1 Resümee der Arbeit

Kurzer Rückblick auf die ursprüngliche Herausforderung und das Ziel der Arbeit. Zusammenfassung der Entwicklung des Test-Frameworks und der wichtigsten Erkenntnisse über das Zusammenspiel von Geometriedichte, Lichtanzahl und der Vielfalt von Zeichenstilen. Abschließende Beschreibung des Nutzens der erstellten heuristischen Entscheidungsmatrix für die Praxis.

7.2 Ausblick und zukünftige Arbeiten

Konkrete Vorschläge für die technologische Weiterentwicklung des Programms. Ideen für den Einbau neuerer Render-Verfahren, um die gefundenen Hardware-Grenzen aufzuheben. Vorschläge für zusätzliche Test-Szenarien mit komplexeren Lichttypen, prozeduralen Materialien sowie für die Untersuchung von Bildfehlern und modernen Methoden der Kantenglättung.

Literatur

- [1] M. Magdics, C. Sauvaget, R. J. García und M. Sbert, „Post-Processing NPR Effects for Video Games,“ in *Proceedings of the 12th ACM SIGGRAPH International Conference on Virtual-Reality Continuum and Its Applications in Industry*, Ser. VRCAI '13, New York, NY, USA: Association for Computing Machinery, 2013, S. 147–156. doi: 10.1145/2534329.2534348.
- [2] T. Harada, J. McKee und J. C. Yang, „Forward+: Bringing Deferred Lighting to the Next Level,“ in *Eurographics 2012 - Short Papers*, The Eurographics Association, 2012. doi: 10.2312/CONF/EG2012/SHORT/005-008.
- [3] K. Fatahalian, S. Boulos, J. Hegarty et al., „Reducing Shading on GPUs Using Quad-Fragment Merging,“ in *ACM SIGGRAPH 2010 Papers*, Ser. SIGGRAPH '10, New York, NY, USA: Association for Computing Machinery, 2010, S. 1–8. doi: 10.1145/1833349.1778804.
- [4] C. Schied und C. Dachsbacher, „Deferred Attribute Interpolation for Memory-Efficient Deferred Shading,“ in *Proceedings of the 7th Conference on High-Performance Graphics*, Ser. HPG '15, New York, NY, USA: Association for Computing Machinery, 2015, S. 43–49. doi: 10.1145/2790060.2790066.
- [5] I. Mallett und C. Yuksel, „Deferred Adaptive Compute Shading,“ in *Proceedings of the Conference on High-Performance Graphics*, Ser. HPG '18, New York, NY, USA: Association for Computing Machinery, 2018, S. 1–4. doi: 10.1145/3231578.3232160.
- [6] O. Olsson, M. Billeter und U. Assarsson, „Clustered Deferred and Forward Shading,“ in *High-Performance Graphics Symposium*, 2012. doi: 10.2312/EGGH/HPG12/087-096.
- [7] C. A. Burns und W. A. Hunt, „The Visibility Buffer: A Cache-Friendly Approach to Deferred Shading,“ *Journal of Computer Graphics Techniques*, Jg. 2, Nr. 2, 2013. Adresse: <https://www.jcgt.org/published/0002/02/04/paper.pdf> (besucht am 31.05.2026).
- [8] J. Thaler, „Deferred Rendering,“ in *Seminar in Computer Graphics*, Vienna, Austria: TU Wien (Vienna University of Technology), 2011. Adresse: https://www.researchgate.net/profile/Jonathan_Thaler2/publication/323357208_Deferred_Rendering/links/5a8fce31aca272140560aaad/Deferred-Rendering.pdf (besucht am 31.05.2026).

- [9] B. Karis, R. Stubbe und G. Wihlidal, „A Deep Dive into Nanite Virtualized Geometry,“ präsentiert auf der ACM SIGGRAPH 2021 Courses (Virtual Event), 2021. Adresse: https://advances.realtimerendering.com/s2021/Karis_Nanite_SIGGRAPH_Advances_2021_final.pdf (besucht am 05. 06. 2026).
- [10] B. Karis, „High Quality Temporal Supersampling,“ präsentiert auf der Advances in Real-Time Rendering in 3D Graphics and Games, ACM SIGGRAPH 2014 Courses, 2014. Adresse: https://advances.realtimerendering.com/s2014/#_HIGH-QUALITY_TEMPORAL_SUPERSAMPLING (besucht am 05. 06. 2026).
- [11] A. Kaplanyan, „CryEngine 3: Reaching the Speed of Light,“ präsentiert auf der SIGGRAPH (Advanced Real-Time Rendering Course), 2010. Adresse: <https://advances.realtimerendering.com/s2010/Kaplanyan-CryEngine3%28SIGGRAPH%202010%20Advanced%20RealTime%20Rendering%20Course%29.pdf> (besucht am 06. 06. 2026).
- [12] T. Akenine-Möller, E. Haines, N. Hoffman, A. Pesce, M. Iwanicki und S. Hillaire, *Real-Time Rendering* (A Balkema Book), Fourth edition, first issued in hardback. Boca Raton, London, New York: CRC Press, 2019. doi: 10.1201/b22086.
- [13] K. Tanaka und T. Komada, „Toon Rendering in Hi-Fi RUSH,“ präsentiert auf der Game Developers Conference, 2024. Adresse: https://media.gdcvault.com/gdc2024/Slides/GDC+slide+presentations/Tanaka_Kosuke_3D_Toon_Rendering.pdf (besucht am 06. 06. 2026).
- [14] R. El Garawany, „Deferred Lighting in Uncharted 4,“ präsentiert auf der SIGGRAPH (Advances in Real-Time Rendering in Games Course) (Anaheim, CA), 2016. Adresse: <https://advances.realtimerendering.com/s2016/> (besucht am 06. 06. 2026).
- [15] T. Sousa und J. Geffroy, „The Devil Is in the Details: idTech 6.66,“ präsentiert auf der SIGGRAPH (Advances in Real-Time Rendering in Games Course) (Anaheim, CA), 2016. Adresse: https://advances.realtimerendering.com/s2016/Siggraph2016_idTech6.pdf (besucht am 06. 06. 2026).
- [16] M. Valient, „Deferred Rendering in Killzone 2,“ präsentiert auf der Guerrilla Develop Conference, 2007. Adresse: https://www.guerrilla-games.com/media/News/Files/Develop07_Valient_DeferredRenderingInKillzone2.pdf (besucht am 06. 06. 2026).
- [17] Muhammad. „Behind the Pretty Frames: PRAGMATA.“ (2026), Adresse: <https://mamoniem.com/behind-the-pretty-frames-pragmata/> (besucht am 06. 06. 2026).
- [18] NVIDIA Corporation, „NVIDIA Turing GPU Architecture Whitepaper,“ NVIDIA, 2018. Adresse: <https://images.nvidia.com/aem-dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf> (besucht am 21. 06. 2026).
- [19] J. Gregory, *Game Engine Architecture*, 3. Aufl. New York: A K Peters/CRC Press, 2018. doi: 10.1201/9781315267845.

- [20] O. Shishkovtsov, „Deferred Shading in S.T.A.L.K.E.R.,” in *GPU Gems 2: Programming Techniques for High-Performance Graphics and General-Purpose Computation*, M. Pharr, Hrsg., Boston, MA: Addison-Wesley Professional, 2005, S. 143–166. Adresse: <https://developer.nvidia.com/gpugems/gpugems2/part-ii-shading-lighting-and-shadows/chapter-9-deferred-shading-stalker> (besucht am 26. 06. 2026).
- [21] A. Lauritzen, „Deferred Rendering for Current and Future Rendering Pipelines,”